

THE FUTURE OF RESEARCH INFRASTRUCTURE

A Vision for Science in the Age of AI



From AI assistants to project consciousness,
collaborative research environments,
and executable science

Copyright © [2026] Axiologic Research

All rights reserved.

KDP publishing rights held by Outfinity SRL (Iași, Romania).

Available for free at www.axiologic.net.

Author's Note

This work was created under the umbrella of Axiologic Research as part of two interconnected research programs, one dedicated to Artificial Intelligence and the other to Outfinitism, both led by Sînică Alboaie. To explore the details of how these two programs intersect and build upon each other, we invite readers to visit the Outfinity Venture Studio page at www.outfinity.ch.

While Artificial Intelligence language models were used to assist with text generation, the foundational philosophy, thematic structure, and analytical approach derive exclusively from the author's decades of dedicated study of social phenomena, social technologies, and genuine hard technologies resulting from various European and self funded research projects. Recently, within the Achilles research project, we have been deeply studying AI generated literature and its automated verification using neuro symbolic approaches; all the free books made available to the public are part of the suite of works we use to test our latest technologies.

Research Note

This edition reorganises the argument around a simple engineering question: if research organisations begin to use AI not as an occasional assistant but as a persistent participant, what infrastructure has to exist underneath the models? The earlier manuscript contained most of the answer, but the answer was distributed across discussions of project memory, agents, provenance, evaluation, compliance, scientific digestion, and automated laboratories. Here those ideas are assembled into one architecture first. The rest of the book then explains why each part is needed, what problem it solves, what it should not be allowed to claim, and how the whole system can be tested rather than merely admired.

The starting point remains the work of Axiologic Research around the ACHILLES project and related experiments with AssistOS, WebMeet, agent capabilities, controlled execution, explicit process representations, provenance, and neuro-symbolic methods. These projects are not presented as a finished stack. They are practical probes into a larger problem: how to preserve scientific continuity, evidence, responsibility, and understanding when machine-generated work becomes abundant and when models, tools, institutions, and even laboratories change during the life of a project.

Terence Tao's 2026 essay *Mathematics in the Age of AI* remains an important intellectual interlocutor. Tao's central move is to ask what research is for once candidate outputs become easier to produce. A proof is not yet mathematical knowledge; it must be checked, explained, digested, accepted, and integrated into the field. This book generalises that diagnosis. In empirical science, code may run while the experiment is poorly designed. A dataset may be genuine while the interpretation is false. A polished manuscript may contain no important contribution. A laboratory

robot may execute exactly the protocol it was given while the protocol itself answers the wrong question. The infrastructure proposed here is designed for the stages after generation as much as for generation itself.

The reader should be able to understand the entire proposal from Chapter 1. The later chapters deepen the argument and make the engineering consequences explicit. The book deliberately avoids the fiction of a single artificial scientist. Its central object is instead a persistent scientific project state around which humans, AI agents, deterministic tools, formal methods, external knowledge, and physical instruments can work under explicit authority. The desired result is not maximal autonomy. It is scientific acceleration without loss of control over what a project means, what it has actually established, and who is responsible for the next consequential step.

Table of Contents

Chapter 1 — The Infrastructure in One Sitting: What We Actually Need to Build

Chapter 2 — Why Existing Research Tools Are Not Enough

Chapter 3 — Scientific Project State: The Substrate

Chapter 4 — From Scientific Intent to Executable Work

Chapter 5 — Agents as Bounded Scientific Contributors

Chapter 6 — Evaluation, Review, and Scientific Digestion

Chapter 7 — Project Memory and Project Consciousness

Chapter 8 — Direction, Values, and Continuous Governance

Chapter 9 — Grounded and Hybrid Research Intelligence

Chapter 10 — When the Infrastructure Reaches the Laboratory

Chapter 11 — A Build and Research Programme

Conclusion — The Instrument We Do Not Yet Have

Chapter 1 — The Infrastructure in One Sitting: What We Actually Need to Build

This chapter is deliberately self-contained. A reader who stops here should still understand the proposed infrastructure, the problem it solves, the boundaries it imposes, and the order in which it should be built. The thesis is concrete: research needs a persistent scientific control plane whose centre is not a model or a chat history but a versioned project state. Humans, AI agents, deterministic tools, external knowledge, and eventually laboratory systems operate around that state through controlled transactions. Generation remains cheap and exploratory; stronger scientific status is earned through evidence, independent evaluation, and explicit authority. The sections that follow move from the change in scientific scarcity, to the architecture, to one complete research transaction, and finally to a realistic build sequence. Later chapters do not replace this picture. They test, qualify, and deepen it.

1.1 The shift from output generation to research control

The book proposes a research control plane: a persistent environment in which scientific questions, claims, evidence, experiments, decisions, agents, tools, and human authority remain connected over time. The need for such an environment appears as soon as AI can generate more plausible scientific work than a team can verify, understand, and integrate. The scarce resource moves from production to judgment.

Imagine a laboratory or research company five years from now, although much of the scenario is already technically possible. A

principal investigator begins the morning with an unresolved question. Before lunch, a literature agent has searched several corpora, a planning agent has proposed four experimental branches, a coding agent has implemented two analyses, a reproduction agent has rerun an older result, and an AI reviewer has identified a statistical weakness in one branch. In parallel, colleagues have discussed the project in a meeting, changed an assumption, and decided that one attractive direction is too expensive to pursue. The organisation has produced more work in a few hours than a small team could once produce in weeks.

Nothing in that picture guarantees scientific progress. The team may now have ten times more artefacts and less clarity about what it actually knows. Two agents may have used different versions of the dataset. A generated experiment may have been designed to optimise a convenient proxy instead of the scientific question. A literature search may have missed a crucial vocabulary. A negative result may be buried in an execution log while the polished report preserves only successful runs. A meeting may have changed the meaning of the project without updating the plan that agents continue to execute. If one model is replaced next month, its conversational memory may disappear and with it the rationale that held the work together.

The problem is not a lack of intelligence in the individual tools. It is the absence of an infrastructure that makes scientific meaning persistent while intelligence is distributed. Contemporary AI products are usually organised around an inference: assemble context, send a request to a model, receive an answer. Scientific projects are organised around a history: a question changes, evidence accumulates, methods fail, interpretations branch, decisions are revised, and responsibility shifts among people and institutions. A research organisation therefore needs an object that survives every individual model call. That object is the project itself, represented as an explicit and versioned scientific state.

The change in scarcity is the deeper reason. Science historically used expensive production as a crude natural filter. Experiments took time, papers required sustained effort, numerical studies consumed scarce compute, and physical prototypes cost money. These filters were never guarantees of quality, but they limited the volume of plausible work that communities had to absorb. AI weakens the production filter. It can draft papers, propose hypotheses, generate code, search combinations, construct diagrams, and prepare analyses at very low marginal cost. The visible outputs of research can therefore become abundant before the slower capacities that turn outputs into knowledge become abundant.

Tao describes the mathematical version as a movement from proof scarcity toward proof abundance. A candidate proof may be cheap while verification, explanation, community digestion, and canonicalisation remain expensive. The same asymmetry appears elsewhere. Drug discovery may have an abundance of candidate molecules while synthesis and clinical evidence remain scarce. Software research may have an abundance of implementations while independent evaluation remains expensive. Social science may have an abundance of plausible narratives while valid measurement and causal identification remain difficult. Automated laboratories may generate enormous numbers of possible experiments while samples, instrument time, safety review, and biological time remain constrained.

The infrastructure problem begins when the output rate exceeds the organisation's scientific metabolism: its capacity to decide what deserves verification, connect claims to evidence, preserve failures, understand disagreements, and turn some results into reusable knowledge. A larger chat window cannot solve that problem because chat is an interaction format, not a scientific memory model. A stronger foundation model cannot solve it by itself because the project must remain meaningful when the model changes. More agents can intensify the problem if they

multiply branches without a stable representation of what those branches mean.

The proposed response is deliberately conservative. It does not require an artificial scientist that owns the research process. It requires a control plane that keeps the process explicit. The environment should know what the project is trying to discover, which claims are provisional, what evidence exists, which experiment produced it, which evaluator checked it, who or what proposed a change, which authority accepted that change, and which uncertainties remain open. Models can then contribute inside bounded roles. Deterministic tools can provide stronger guarantees where appropriate. Humans and institutions can retain authority over scientific meaning, risk, publication, and physical action.

Table 1. The minimum build: concrete responsibilities and an implementable first form.

Responsibility	What it must do	Practical first implementation
Scientific Project State	Keep stable identities, versions, status, dependencies, and access metadata for the objects that determine scientific action.	Event log plus relational or graph index, linked to existing repositories and object storage.
Transaction and orchestration	Turn intentions into bounded plans, branches, executions, reviews, and explicit commit decisions.	Transaction service with workflow hooks and model-assisted planning.
Agent and tool runtime	Execute models, code, retrieval, simulations, and tools with isolation, resource limits, and receipts.	Container sandboxes, typed adapters, model gateways, and capability-scoped credentials.

Responsibility	What it must do	Practical implementation first
Evidence and evaluation	Attach independent checks to claims, experiments, code, and sources without collapsing them into one score.	Common evaluation envelope for reproduction, citations, statistics, novelty, formal tools, and human review.
Governance and authority	Separate what a component can do from what it is permitted to change or cause.	Role and capability policy, data classification, approval gates, and auditable external actions.
Research workspace	Let researchers interact through documents, code, meetings, notebooks, diagrams, and conversational access.	Web-based project workspace that renders and edits the shared scientific state.
Project memory and later intelligence	Recover context, alternatives, direction, and long-term learning from authorised history.	Begin with state queries and event summaries; add Project Consciousness only after the history is trustworthy.

Table 1 makes the proposal intentionally concrete. The infrastructure is not defined by one model provider or one database. It is defined by responsibilities that have to exist somewhere in the system. A small implementation can combine several responsibilities in one service. A larger consortium may distribute them across organisations. What matters is that the boundary between state, execution, evaluation, and authority is real enough to inspect and enforce.

The table also reveals what the proposal is not. Project Consciousness, for example, is not the database. It is a higher-level service that interprets versioned history. A novelty agent is not the authority that decides whether a research programme is valuable. It is an evaluator that prepares evidence

for a decision. A laboratory controller is not a conversational agent with broad permissions. It is a bounded execution interface under lower-level safety constraints. These distinctions prevent attractive concepts from becoming vague labels for the entire system.

The rest of the book can therefore be read as an argument for one architectural principle: generation should be easy, but scientific commitment should be explicit. A system may allow broad exploration in provisional branches while requiring evidence, independent checks, and appropriate authority before a claim changes status, a result is published, restricted data move, or a physical experiment begins. Speed is gained in the abundant stages. Restraint is concentrated where error becomes expensive.

This principle is also compatible with scientific freedom. The environment does not need to formalise every thought. Brainstorming can remain informal. Natural language can remain the primary medium for many questions. Alternative hypotheses can coexist. The purpose of explicit state is not to force science into one ontology; it is to make the small set of relations that determine action and accountability persistent enough that humans and machines can coordinate without continuously reconstructing meaning from prose.

If the reader remembers only one idea from this section, it should be that the future Research IDE is not primarily an editor with an AI assistant. It is a scientific control plane with familiar editors, notebooks, meetings, and agents on top. Its most important product is not another answer. Its most important product is a project that can still explain what it believes, why it believes it, what remains uncertain, and what is authorised to happen next after thousands of machine contributions have passed through it.

1.2 The architecture: a scientific control plane

The proposed architecture has a persistent Scientific Project State at its centre. A Research Workspace lets people interact with that state. A transaction and orchestration layer turns intentions into controlled plans. An Agent and Tool Runtime performs work in sandboxes. An Evidence and Evaluation layer challenges outputs. A Governance and Authority layer determines which proposed changes may become committed scientific state or external action. Models are replaceable participants; the project state is the continuity mechanism.

The easiest way to misunderstand the proposal is to imagine a monolithic application called the Research IDE that contains every future feature. The more useful interpretation is closer to an operating system or control plane. The infrastructure defines a small number of contracts that allow heterogeneous components to work on one scientific project without erasing their different guarantees. A language model can interpret an ambiguous request. A database can retrieve exact records. A statistical package can compute an estimate. A solver can verify a formal property. A human can decide whether the result answers the scientific question. The system becomes trustworthy not because one component is universally intelligent but because the relation among these forms of work remains visible.

A scientific control plane: models are replaceable; project state is persistent

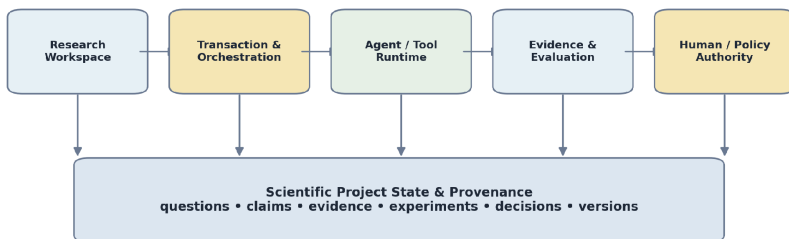


Figure 1. The research infrastructure as a scientific control plane. Project state persists while models and tools remain replaceable.

The diagram is deliberately simple because the architecture should be explainable on a whiteboard. The Research Workspace is the human-facing surface. It contains familiar forms of work: documents, references, notebooks, code, datasets, figures, meetings, experiments, and conversational access. The workspace is not the source of truth. It renders and edits scientific objects maintained by the project state. A paragraph in a manuscript can therefore be linked to a claim. A figure can be linked to the experiment run that produced it. A meeting decision can propose a state change rather than becoming authoritative merely because it appeared in a transcript.

Scientific Project State and Provenance form the backbone. A minimal state layer gives stable identities and versions to questions, hypotheses, claims, evidence, methods, experiments, evaluations, decisions, constraints, and important artefacts. It records relations such as supports, contradicts, depends on, supersedes, produced by, evaluated by, and accepted by. Large binary objects do not need to live inside one graph database. Code can remain in Git, data in an object store, papers in a document system, and instrument logs in a laboratory platform. The state layer preserves the scientific meaning that connects those stores.

The Transaction and Orchestration layer turns a request into controlled scientific work. It is responsible for planning, branching, selecting capabilities, assigning budgets, handling retries, observing dependencies, requesting evaluation, and escalating decisions. It should not be a scientific oracle. For some tasks the plan can be a fixed workflow. For others it can be constructed dynamically by an agent. The stable requirement is that the chosen plan, inputs, assumptions, and authority boundaries become inspectable before or during execution

rather than remaining hidden inside one model's internal reasoning.

The Agent and Tool Runtime is where intentions become operations. It manages model calls, coding agents, retrieval services, simulations, data mounts, credentials, local programs, external APIs, and eventually laboratory adapters. Every run should have an identity, declared inputs, environment metadata, resource consumption, outputs, and failure state. Sandboxing belongs here because a model that can read a hostile repository should not inherit unrestricted network access or project secrets. The runtime is also where model replacement becomes practical: the same scientific role can be reimplemented with a different model while preserving the role's input, output, evidence, and policy contract.

Evidence and Evaluation are deliberately separated from generation. A citation checker, reproduction service, statistical validator, formal solver, contradiction search, novelty analysis, and human review are not interchangeable, but they can share a common envelope. Each evaluator identifies its target, method, inputs, output, uncertainty, version, and scope. Several evaluators can disagree. The infrastructure should preserve the disagreement rather than average it into a single confidence score. Evaluation produces a review case; it does not automatically produce truth.

Governance and Authority decide what consequences a component may create. Capability and authority must be separated. An agent may be able to write code but not authorised to execute it against confidential data. A literature agent may search public corpora but not disclose an unpublished hypothesis to an external service. A meeting agent may draft a project decision but not commit it. A laboratory planning agent may propose a protocol but not move a robotic arm. Human and institutional roles, deterministic policies, data classifications,

safety boundaries, budgets, and approvals therefore become machine-readable parts of the research environment.

These layers communicate through typed project objects and research transactions rather than through informal agent conversations alone. This matters because agent-to-agent chat is a poor systems boundary. If one agent tells another that a result has been verified, the receiving agent must trust the wording, understand the scope, and infer the authority. If the evaluation exists as an object with a known method and version, the receiving component can reason about the exact status. Structured exchange reduces the amount of scientific meaning that has to be reinvented in every prompt.

Table 2. Stable responsibilities matter more than the technology chosen to implement them.

Layer	Stable responsibility
Workspace	Present project objects in familiar research interfaces and make machine participation visible.
Scientific state and provenance	Preserve identity, status, versions, evidence relations, access metadata, and authoritative history.
Orchestration	Plan, branch, budget, stop, coordinate capabilities, request evaluation, and escalate decisions.
Runtime	Execute models and tools in controlled environments and capture consequential actions.
Evaluation	Challenge outputs through methods whose scope and failure mode remain explicit.
Governance and authority	Enforce permissions, data rules, approvals, safety boundaries, and external-action thresholds.

The table separates responsibility from implementation. A small prototype might use a relational database for state, an event log for transitions, S3-compatible storage for large artefacts, Git for code, container sandboxes for execution, and model APIs behind role-specific wrappers. None of those technologies is the thesis. The thesis is that the system must preserve the contracts even if the implementation later changes. This prevents the research programme from confusing its current prototypes with permanent architectural truth.

The same separation clarifies the position of current Axiologic work. AssistOS and the ACHILLES IDE explore the workspace and project-centred continuity. WebMeet explores the shared collaboration surface. AchillesAgentLib and related agent work explore reusable capabilities. Ploinky explores controlled deployment and isolation. MRP-VM and SOP Lang experiments investigate explicit process representations and the boundary between language-mediated proposals and inspectable execution. OpenDSU explores identity, version continuity, and controlled sharing across organisational boundaries. These lines can converge without requiring every current component to survive unchanged.

A good architecture should remain useful even if some research hypotheses fail. Suppose Project Consciousness proves unreliable as a narrative service. Persistent project state and provenance remain valuable. Suppose a symbolic execution layer creates excessive authoring overhead. Controlled agents and explicit evaluation still matter. Suppose foundation models become dramatically more reliable. Versioning, permissions, reproducibility, and human responsibility do not disappear. The architecture earns credibility by making speculative components replaceable.

The practical product vision follows naturally. At the surface, researchers can continue to use documents, chat, code, meetings,

notebooks, and domain visualisations. Underneath, those interfaces operate on the same project state and authority model. The researcher does not need to become a database engineer. Formal structure appears progressively where it improves reproducibility, control, or explanation. A document can remain a document until a claim requires evidence tracking. A conversation can remain exploratory until the group decides to create an experiment. A generated analysis can remain provisional until independent evaluation justifies promotion.

This layered design is less glamorous than a universal artificial scientist, but it is a stronger foundation for scientific organisations. It lets capability grow without requiring trust to grow at the same speed. The system can grant broader autonomy to roles that have earned it, preserve narrow authority for high-risk actions, and replace models without resetting the project. In that sense, the control plane is not an obstacle to more capable AI. It is the infrastructure that allows more capable AI to become scientifically usable.

1.3 The research transaction: from question to accepted state

The basic unit of work is a research transaction. A participant reads a known project state, proposes a change, plans and executes bounded work, collects evidence, submits the result to independent evaluation, and changes the authoritative state only when the configured human or institutional authority accepts the transition. Failed attempts remain part of history without being confused with scientific commitment.

A concrete architecture becomes easier to understand when followed through one complete episode. Consider a research team asking whether a new regularisation method improves the robustness of a machine-learning model under a specific distribution shift. In an ordinary assistant-centric workflow, the

researcher might ask a model to review literature, generate code, run an experiment, and summarise the result. The final answer could be useful, but the scientific organisation of the work would remain mostly implicit. A research transaction makes each consequential transition explicit without requiring the researcher to manually fill in administrative forms.

The transaction begins from a state version. The current project already contains the question, the baseline model, the accepted dataset version, earlier experiments, known constraints, and perhaps a decision that robustness under this shift is strategically important. A planner receives an authorised view of that state. It may use a language model to propose an experiment, but the proposal becomes a candidate plan object. The plan identifies the code revision, datasets, metrics, comparison baselines, expected resource cost, and what result would count as relevant evidence. If the scientific question is ambiguous, the plan should preserve the ambiguity or ask for clarification rather than silently narrowing it.

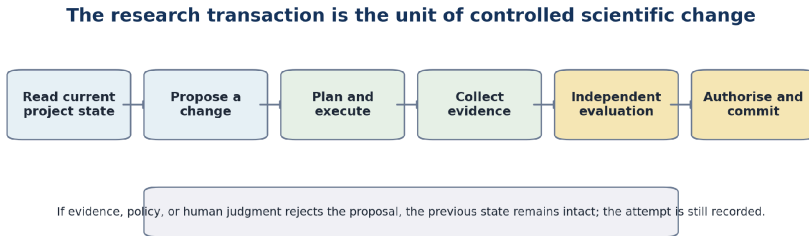


Figure 2. The research transaction is the unit of controlled scientific change.

The execution phase is deliberately separated from the state change. A coding agent can create a branch, modify the implementation, and run the experiment in a sandbox. The runtime records environment versions, parameters, resource consumption, outputs, and failures. If the code fails to run, the transaction has an operational failure. That failure does not

weaken the scientific hypothesis. If the experiment runs and shows no improvement, the result is a negative scientific observation rather than an agent failure. This distinction is essential because research automation otherwise tends to collapse operational and epistemic outcomes into one success flag.

Evidence is then assembled. The result may include raw metrics, confidence intervals, plots, execution receipts, code references, and links to the exact data version. A reproduction role can rebuild the environment and rerun the experiment independently. A statistical evaluator can check whether the comparison is supported by the design. A literature or novelty role can determine whether the method is already known under another name. A method reviewer can challenge whether the selected distribution shift is meaningful. These evaluators need not agree, and their disagreement is itself a scientific state.

Only after evaluation does the transaction ask whether the project should change its authoritative state. A human researcher may accept the evidence and promote a claim from candidate to supported. The team may instead keep the claim disputed, request another experiment, or reject the interpretation while preserving the raw result. A project policy might allow routine computational results to be committed automatically when independent checks pass, while requiring explicit approval for claims intended for publication. The system records both the action and the authority under which it occurred.

The transaction model solves several problems at once. It gives concurrency a meaning. Two agents can start from the same state and propose incompatible changes without silently overwriting one another. The system can preserve both branches and ask for reconciliation. It gives rollback a scientific meaning. If a dataset is later found to contain an error, dependent claims can be marked for reconsideration without deleting the historical

fact that they were once accepted under the previous state of knowledge. It also gives provenance a natural capture point because the transaction already knows the actors, inputs, tools, environment, outputs, and decision.

Table 3. A research transaction can produce several scientifically meaningful outcomes.

Outcome	Meaning for the project
Committed positive evidence	Execution and review support a state change accepted under the required authority.
Negative scientific result	The experiment or analysis ran adequately but did not support the candidate claim.
Operational failure	The intended scientific test was not completed because code, data, environment, or equipment failed.
Disputed result	Evidence exists, but evaluators or humans disagree about interpretation or adequacy.
Blocked action	A policy, rights, safety, or authority condition prevents execution or external consequence.
Provisional retention	The result may be valuable but has not earned mature authority or sufficient human understanding.

The table is useful because it prevents a common category error in agentic systems: assuming that every completed task should end in a positive update. Research transactions can end with valuable non-commit outcomes. An experiment can show that the proposed effect is absent. A novelty search can reveal that the central idea is known. A compliance check can block a data transfer while suggesting a safe alternative. A reproduction attempt can remain unresolved because an external dependency is missing. Preserving these outcomes makes the organisation learn from failure rather than hiding it behind retries.

The same transaction pattern scales beyond computational experiments. In literature review, the proposed change might be a new claim-evidence relation. In a manuscript, it might be a revised interpretation. In a meeting, it might be a decision to suspend a branch. In compliance, it might be approval for a specific data action under a defined policy version. In a laboratory, it might be an approved experiment plan whose execution is delegated to instrument controllers but whose scientific interpretation remains a later transaction.

The transaction also creates an explicit boundary for external actions. Publishing a paper, sending an email to an external partner, purchasing a large compute allocation, transferring restricted data, or issuing commands to physical equipment are not merely tool calls. They change the world outside the project and may be irreversible. The architecture can therefore require a higher authority threshold than it uses for reversible sandbox work. The user experience can remain simple, but the underlying distinction must be enforceable.

A research transaction is not intended to bureaucratised every thought. Most exploratory language need not become a transaction. The transition occurs when work creates or changes a persistent scientific object, consumes controlled resources, crosses a rights boundary, or makes an external commitment. This is similar to version control in software: developers do not commit every keystroke, but meaningful changes acquire identity and history. Scientific work needs an equivalent level of disciplined granularity.

The transaction model is also a falsifiable part of the programme. It may turn out that a rich semantic transaction is unnecessary for many tasks and that ordinary event sourcing plus lightweight metadata captures enough value. That would be a useful result. The architectural claim is not that every research act requires a complex schema. It is that the organisation needs a stable

mechanism for distinguishing proposals, executions, evidence, evaluations, and accepted changes. The smallest mechanism that reliably preserves that distinction should win.

Once this unit exists, many other concepts in the book stop floating. Project Consciousness observes transactions and their history. Versioning records their branches. Evaluation attaches to their outputs. Governance controls their consequences. The Research Workspace visualises their state. Agents participate through bounded roles. The automated laboratory executes a specialised form of transaction with stronger safety gates. What appeared to be a collection of concepts becomes one system with a recurring operational rhythm.

1.4 What must be built first, and what can remain research

The near-term build should be smaller than the complete vision: persistent scientific state, versioned provenance, a transaction engine, bounded agent execution, independent evaluation hooks, and a usable research workspace. Project Consciousness, sophisticated direction intelligence, cross-institution federation, and automated laboratory control can then be developed as research layers. The architecture should earn complexity through measured benefit.

A vision becomes credible when it can say what ten engineers should build first. The minimum viable research infrastructure is not a universal ontology, a fleet of dozens of agents, or a robotic laboratory. It is a project-centred environment that can follow one real research episode from intent to evidence to review while preserving state and authority. If that cannot be made useful, the more ambitious layers have no foundation.

The first engineering target is a minimal Scientific Project State. It should contain only objects that improve action or evaluation. Questions, claims, evidence, experiments, decisions, constraints, evaluations, and artefacts are enough to begin. Each object needs

a stable identity, version, actor, status, provenance links, and access metadata. The implementation can combine an event store for changes, a database for indexed relations, ordinary repositories for code and documents, and an object store for large data. The project state does not need to contain the world. It needs to make the current research situation reconstructable.

The second target is a transaction engine tied to controlled execution. A user or agent should be able to propose a change against a known state, invoke a capability, run tools in a sandbox, collect outputs, request specified checks, and ask for commit under policy. This engine is where model calls, container execution, permissions, and evidence capture meet. It is also where a system can begin to distinguish a draft from an authoritative project change.

The third target is a small capability registry. Instead of treating agents as characters, the platform should know that a particular implementation performs literature retrieval, code reproduction, citation checking, experiment planning, or another bounded role. The registry records accepted inputs, produced outputs, required tools, data sensitivity limits, estimated cost, abstention conditions, and evaluators required before the output can be committed. The first version can be manually authored. Later work can investigate how successful improvisations become qualified reusable capabilities.

The fourth target is an evaluation envelope. A project should be able to attach multiple evaluations to the same object without collapsing them into one score. A reproduction result, citation check, statistical assessment, novelty comparison, and human review should all identify their method and scope. This creates immediate value even before the evaluators become sophisticated because the system can stop using ambiguous words such as verified as if they had one universal meaning.

The fifth target is the workspace. Researchers must be able to use the system without thinking in database operations. Documents, code, notebooks, figures, meetings, and conversational assistance should remain familiar. The new structure appears when useful. A researcher can ask why a claim is believed and receive evidence links. A manuscript can indicate that a paragraph has become stale because its underlying claim changed. A meeting can propose a decision and show that it still awaits approval. A project briefing can summarise state changes rather than message volume.

Table 4. Foundation first: what should be engineering commitment and what should remain research.

Layer	Status in a credible programme
State, provenance, transactions, and sandboxed execution	Engineering foundation that must work on real projects before stronger claims are made.
Evaluation envelope and capability registry	Near-term engineering plus empirical qualification of individual roles.
Project Consciousness and direction intelligence	Research hypotheses built above reliable project history and evaluated for distortion, utility, and overload.
Federated cross-institution research objects	Later interoperability layer, preserving local policy and data control.
Automated laboratory supervision	Later high-consequence extension, beginning with adapters to existing facilities and native safety systems.

The table distinguishes engineering foundation from research ambition. This separation is important for programme management. A project can promise that the state and transaction substrate will be built, while treating Project Consciousness or automated direction selection as hypotheses to test. This avoids a common failure in ambitious AI projects where speculative components are described as if their usefulness has already been demonstrated.

Project Consciousness belongs in the second wave because it depends on trustworthy history. A system cannot reconstruct why a project changed if the underlying events, branches, evidence, and decisions were never captured. Once the foundation exists, the hypothesis becomes testable: can a mostly passive AI reconstruct decisions accurately, preserve alternative interpretations, recover abandoned ideas when conditions change, and do so without turning project memory into surveillance? A persuasive but unsupported narrative should count as failure.

Direction intelligence also belongs above the foundation. Novelty assessment, significance, learning value, risk, cost, and strategic fit are important, but they are dangerous when compressed into an oracle. A direction service should therefore consume explicit project state and produce challengeable comparisons. It can become more useful as the project accumulates internal history. Its success should be measured by better decisions and saved scientific effort, not by the elegance of its rankings.

Cross-institution federation is another later layer. The architecture should be designed so that partners can exchange typed research objects, receipts, and bounded capabilities without centralising all data. But the first prototype does not need to solve every privacy and interoperability problem. It needs clean identities, provenance, and access boundaries so that federation can be added without rewriting the scientific core.

Automated laboratory integration should come later still because physical action raises the cost of architectural mistakes. A near-term laboratory prototype should be supervisory rather than universal: connect one or two existing automated facilities through adapters, preserve native safety systems, record approved protocols and actual execution, and test whether the research environment improves traceability or decision quality. The goal is not to demonstrate that an LLM can operate a robot.

The goal is to show that layered scientific control survives contact with the physical world.

The programme should also remain willing to simplify. If researchers find that explicit claim objects add little value in a particular domain, the state model should be reduced. If an LLM-based memory service introduces confident distortions, its authority should be narrowed to retrieval. If a symbolic workflow adds more maintenance than reliability, it should be used only where the guarantees justify the cost. Infrastructure should grow from measured necessity rather than from the desire to display architectural sophistication.

This restraint is part of the scientific character of the proposal. The system is not offered as an inevitable future. It is a set of hypotheses about what becomes necessary when machine contribution grows. The most important hypothesis is that explicit project state, provenance, bounded execution, independent evaluation, and authority create compounding value over long research episodes compared with folders, chats, repositories, and ad hoc assistants. That claim can be tested with real projects, correction episodes, model replacements, onboarding tasks, and adversarial cases.

If Chapter 1 is the only chapter a reader has time to read, the proposal should now be clear. Build a persistent scientific state. Make meaningful work pass through controlled transactions. Let agents and tools operate as replaceable capabilities with explicit authority. Keep evaluation independent and scoped. Preserve human and institutional responsibility for consequential transitions. Put familiar research interfaces on top. Then investigate richer memory, direction support, federation, and laboratory autonomy as layers whose value has to be earned. The remaining chapters explain why each of these choices follows from the changing nature of science in the age of AI.

Chapter 2 — Why Existing Research Tools Are Not Enough

Having stated the architecture, the next question is whether such a layer is actually necessary. Research organisations already possess excellent document editors, repositories, notebooks, data platforms, workflow engines, reference managers, collaboration tools, and AI assistants. The problem is not that these tools are primitive; it is that each usually owns only a local artefact or activity. The scientific relations that make a project coherent remain largely implicit in human memory. This chapter examines why AI makes that gap operationally expensive, why abundance relocates rather than removes scarcity, and why project continuity must survive changes of model, personnel, and institution. The argument is deliberately comparative: the new infrastructure should complement existing tools and justify every additional layer by a problem they do not already solve well.

2.1 From files and chat to scientific state

Research software already manages documents, code, datasets, notebooks, workflows, references, and communication. What it usually does not manage is the scientific meaning that connects them. AI exposes this gap because agents must infer, on every task, which artefact is authoritative, which hypothesis is active, which experiment invalidated a claim, and which decision superseded an earlier one.

A contemporary research project can be technologically sophisticated while remaining semantically informal. The manuscript lives in a document editor. Code lives in Git. Experimental results live in notebooks and storage buckets. References live in a citation manager. Tasks live in a project-management system. Important decisions live in meeting minutes, email, or the memories of senior researchers. Each tool

can perform its local job extremely well. The scientific project emerges only in the minds of the people who understand how the pieces relate.

For human teams this arrangement is inconvenient but survivable because researchers continuously repair context through conversation. They remember that a figure in the manuscript came from an older run and should not be trusted. They know that an apparent negative result was caused by an instrument problem. They remember that a branch was abandoned for budget reasons rather than because the hypothesis was refuted. They know which spreadsheet is current even when several files have similar names. Much scientific coordination consists of tacit semantic integration.

AI agents do not inherit that tacit layer automatically. An agent asked to update a report must reconstruct the project from accessible artefacts. Retrieval can find relevant text, but relevance is not authority. A recent message can describe a tentative idea while an older decision remains accepted policy. A notebook can contain exploratory code that was never validated. A manuscript can lag behind the dataset. A vector search system can retrieve semantically similar passages without knowing which one was superseded. As agent activity increases, the cost and risk of repeatedly inferring scientific state from raw artefacts increase with it.

The solution is not to put every research object into a rigid universal database. Scientific fields differ too much, and premature formalisation can freeze a bad representation. The solution is to identify a small semantic core that matters across domains. Questions tell the project what it is trying to resolve. Claims express propositions whose status can change. Evidence points to observations, sources, calculations, or validated analyses. Experiments bind methods to inputs and environments. Decisions record authoritative changes in direction. Constraints

represent safety, legal, ethical, resource, or scientific limits. Evaluations challenge objects without overwriting them. These are not a theory of science; they are an operational grammar for coordination.

A project state also gives artefacts stable context. A document is no longer merely the latest file. It can be a view over claims and evidence at a particular project version. A figure can identify the run that generated it. A dataset can declare which analyses depend on its current version. A meeting decision can be linked to the evidence considered at the time. When an object changes, dependent parts of the project can be marked for reconsideration rather than silently drifting out of sync.

Table 5. Existing research tools are strong locally but leave scientific relations mostly implicit.

Tool category	What remains outside its normal responsibility
Document editor	Whether a claim is current, which evidence supports it, and which experiment made the paragraph stale.
Git repository	What a branch means scientifically and why a merge was accepted.
Notebook or data platform	Whether an output is valid evidence for the project claim and which decision depends on it.
Reference manager	Which source span supports a specific claim and whether later correction changes project state.
Workflow engine	Whether task completion answered the scientific question rather than merely completed operations.
Chat assistant	Which statements are provisional, authoritative, superseded, or tied to persistent project objects.

The table clarifies why the proposed infrastructure complements existing tools instead of replacing them. Git remains excellent at file history. A notebook remains excellent for interactive computation. A reference manager remains excellent at organising literature. The missing layer is the scientific relation among those systems. The Research IDE should therefore integrate rather than imitate them, preserving their native strengths while exposing typed research objects and state transitions above them.

This is particularly important for model replacement. A long research project may outlive several generations of foundation models, agent frameworks, or vendors. If the project's rationale exists mainly in one assistant's long-term memory, replacing the model can create a semantic reset. If the project has explicit questions, claims, evidence, decisions, and provenance, a new model can reconstruct an authorised view of the same state and its understanding can be tested against that state. Model competition then becomes compatible with scientific continuity.

The state layer also creates a better context compiler. Large projects can contain millions of files, source spans, experiment logs, and evidence relations. No agent should receive all of them. A role can ask for the subset relevant to a task, and the platform can return a view that preserves access rules, versions, and provenance. Semantic retrieval remains useful, but it operates inside a stronger frame of exact identifiers and structured relations. The result is less context, not more, and therefore often better model behaviour.

A scientific source of truth should not be confused with one official opinion. Science legitimately contains disagreement. The state must therefore support competing claims, alternative branches, and unresolved evaluations. Its function is to make authority explicit, not to force consensus. It can record that one dataset version is current, that one claim is provisional, that

another has been contradicted, and that two theories remain live. Researchers can disagree about interpretation without also disagreeing about which artefacts and decisions constitute the current project history.

The design challenge is to find the smallest useful structure. Too little structure leaves agents reconstructing meaning from prose. Too much structure burdens researchers and creates brittle ontologies. The right answer will vary by domain and should be investigated empirically. A strong platform makes schema evolution normal: projects can add domain-specific object types and validators while the core ideas of identity, version, provenance, status, and authority remain stable.

This shift from files and chat to scientific state is therefore less radical than it first appears. Research teams already maintain such a state mentally. The proposal is to externalise only the parts that increasingly many humans and machines must share. The value is not bureaucratic completeness. It is the ability to return six months later, replace the model, inspect why the project believes what it believes, and continue without rebuilding the intellectual history from fragments.

2.2 Abundance moves the bottleneck downstream

AI does not abolish scarcity in science; it relocates it. Candidate generation becomes cheap while verification, physical experimentation, explanation, expert review, ethical judgment, and integration into shared knowledge remain expensive. Infrastructure should therefore optimise scientific metabolism rather than raw output.

The intuitive story of automation is that a faster tool accelerates the same pipeline. If writing a draft becomes ten times faster, perhaps research becomes ten times faster. That inference is correct only when writing is the bottleneck. In scientific work,

the bottleneck moves. Once drafts are cheap, the scarce resource may become expert review. Once candidate molecules are cheap, synthesis and toxicology dominate. Once code generation is cheap, reproducible evaluation and security become relatively more expensive. Once hypotheses are cheap, deciding which experiment deserves a scarce sample becomes the hard problem.

This shift changes what infrastructure should measure. A system that celebrates the number of generated reports can be productive by its own metric while making the organisation worse. More reports require more reading, create more contradictory claims, and increase the probability that weak evidence will be mistaken for progress. The correct question is not how much the system produces but how efficiently the organisation converts production into tested, understood, reusable knowledge.

The term scientific metabolism is useful because research is not a one-way production line. A result enters the project, is challenged, compared, explained, sometimes rejected, sometimes revived, and occasionally integrated into a stable method or shared theory. Negative results can prevent future waste. Reviewer objections can change the question. A new source can reopen an old branch. Knowledge circulates and is transformed rather than simply accumulated.

Tao's discussion of proof indigestion gives a precise example. If proofs can be generated faster than mathematicians can verify and understand them, a field can become saturated with formally interesting objects that consume attention without becoming part of the discipline's usable knowledge. The same phenomenon can occur in any research organisation. Agents can explore dozens of branches, but if nobody knows which branch is credible, the organisation has created possibility without orientation.

The economics of error also change. When a paper requires months of work, reviewers can assume that some effort preceded submission, even though that assumption is imperfect. When a polished manuscript can be produced cheaply, the volume of superficially plausible low-value material can grow far faster than volunteer review capacity. The average quality of generated text could improve while the total burden of filtering still becomes unsustainable. Infrastructure must therefore move checking earlier and make evidence easier to inspect.

The same problem appears inside a company. Suppose twenty agents can generate alternative architectures for a technical problem. The organisation may spend more human time understanding the alternatives than it would have spent designing three options itself. Agentic productivity becomes negative unless the system can detect duplication, expose assumptions, compare branches, and route attention to the choices where judgment matters. Coordination becomes an epistemic problem, not just a scheduling problem.

Table 6. AI relocates scarcity rather than removing it.

Where candidate generation becomes cheap	Where scarcity can move
Mathematical proofs and derivations	Verification, exposition, expert review, and canonicalisation.
Candidate molecules and protocols	Synthesis, samples, safety, biological time, and clinical evidence.
Generated software and analyses	Reproducibility, security, methodological evaluation, and trustworthy data.
Research hypotheses and manuscripts	Expert attention, discriminating experiments, significance, and community digestion.
Automated experiment plans	Instrument time, physical resources, safety authority, and interpretation of observations.

The table is not meant to predict one universal future. Different fields move different bottlenecks. The common pattern is that the scarce resource becomes the stage that AI has not commoditised. This is why a research infrastructure cannot be designed around one benchmark of model capability. It needs to make the local bottleneck visible and to preserve the chain from candidate generation through evidence, explanation, review, and decision.

Attention becomes a first-class resource. Not all unresolved issues deserve equal expert time. A claim on which many other conclusions depend may deserve review before a novel but isolated result. A cheap experiment that distinguishes two major hypotheses may deserve priority over a precise measurement that changes nothing. A compliance ambiguity before an irreversible data transfer deserves immediate human review, while routine citation checks can be automated. The system should help allocate attention without pretending that attention can be optimised by one universal score.

This is also where negative results become economically valuable. When generation is cheap, rediscovering the same dead end becomes easy. A project that preserves failed experiments, rejected interpretations, and exhausted searches can prevent many agents from repeating attractive mistakes. The history of what did not work becomes part of the organisation's scientific capital. Current publication incentives often discard that capital; a project-centred infrastructure can retain it privately and expose it selectively.

Abundance can also weaken education. A junior researcher who can generate a correct analysis without understanding why it is correct may produce output while failing to develop judgment. A future environment should therefore distinguish production mode from learning mode. In some contexts the system should expose the difficult step, ask for an explanation, or require the researcher to inspect an assumption instead of smoothing every

problem into a finished answer. Scientific capacity is not only the output of the current project; it is also the competence the organisation retains after the tools change.

The downstream bottleneck therefore provides a design test for every proposed feature. Does the feature create more material to inspect, or does it help the project decide what deserves inspection? Does it hide uncertainty behind polished prose, or make uncertainty easier to locate? Does it increase autonomous activity while leaving authority ambiguous, or does it reserve human attention for consequential decisions? The infrastructure should make generation abundant and commitment selective.

This perspective also tempers predictions about autonomous science. Even if AI systems become capable of producing research-level results, science remains an institutional process in which evidence, explanation, replication, responsibility, and collective acceptance matter. The bottleneck may move again as tools improve, but the need for an environment that can represent those stages remains. A project that knows where its scarcity lies is better positioned to benefit from capability improvements than a project that treats every additional output as progress.

2.3 Project continuity must survive models, people, and institutions

Research projects often live longer than particular models, software stacks, team memberships, funding periods, or institutional arrangements. The infrastructure must therefore preserve a durable identity for the project and enough history to reconstruct meaning after these elements change.

Long research programmes experience a quiet form of decay. People leave. Vendors disappear. Model versions are retired. Repository structures change. Datasets are corrected.

Terminology evolves. A decision that was obvious during a meeting becomes mysterious a year later. The final reports survive more reliably than the reasons that produced them. This loss is tolerable when projects are small and human memory is strong. It becomes dangerous when large numbers of agents make decisions and modifications that no individual person observes in full.

The ordinary solution is documentation, but documentation usually captures conclusions rather than state transitions. A report explains what the team now believes. It rarely preserves every serious alternative, the evidence that changed the decision, the limitation that made an experiment inconclusive, or the reason a method was temporarily accepted. Meeting transcripts contain more detail but too little structure. Raw agent logs contain even more detail but are usually impossible to read. The project needs a middle layer that preserves meaning without requiring complete surveillance.

Versioning is one part of the answer. Software engineering learned that complex collaborative work requires recoverable history. Research needs the same property for more than files. A scientific branch can represent an alternative hypothesis, a different statistical interpretation, a corrected dataset, or a new protocol. A merge can represent a scientific decision about which parts are compatible. A fork can allow independent reproduction without modifying the original project. These operations make intellectual alternatives durable rather than forcing premature convergence.

Provenance adds causality. A result should identify the sources, data, tools, transformations, actors, and approvals that produced it. Provenance does not prove truth. A perfectly traceable experiment can still be badly designed. Its value is that later correction becomes localisable. If a dataset is withdrawn, the system can identify affected analyses. If a model version is found

to have a systematic failure, the project can locate evaluations that depended on it. If an external service was compromised, affected outputs can be quarantined.

A durable project identity also protects against model lock-in. The project should not become whatever the current copilot remembers. A new model should receive a structured view of questions, claims, evidence, decisions, and unresolved issues. Its ability to understand that view can be evaluated. The organisation can compare models on the same role and state instead of migrating a whole project into a new conversational culture each time the market changes.

Institutional boundaries make continuity harder. A consortium may include a university, a hospital, a company, and an external laboratory. They cannot necessarily share one memory store or one data policy. A partner may provide an evaluation without exposing the underlying dataset. Another may allow a computation to run locally but prohibit export. The shared project therefore needs bounded objects and receipts that can cross organisational boundaries while internal details remain protected.

Table 7. Continuity is provided by several distinct layers, not by one memory feature.

Layer	Primary question it answers
Version control	What changed and which alternatives or forks existed?
Provenance	Where did this object come from and which transformations produced it?
Scientific project state	What do these versions mean for current questions, claims, experiments, and decisions?
Project memory	Which events and rationales are important enough to preserve for future work?

Layer	Primary question it answers
Project Consciousness	How can authorised history be reconstructed and queried without pretending interpretation is fact?
Authority record	Who or what was entitled to accept the consequential transition?

The table shows why versioning, provenance, memory, and Project Consciousness should not be collapsed. Version control preserves change. Provenance records derivation. Project memory preserves selected scientific events. Project Consciousness interprets that history. Human and institutional authority determine what the interpretation is allowed to influence. Each layer answers a different question and therefore has a different failure mode.

Continuity also has a temporal dimension. A claim can be historically authentic and no longer currently acceptable. An analysis may have been reasonable given the literature available in 2026 and become obsolete after later evidence. A legal permission can expire. A model can be valid for one data distribution and fail after conditions change. The infrastructure should preserve what the project knew at the time while also marking what later evidence changes. This makes retrospective evaluation fairer and current action safer.

The need for continuity is one reason the infrastructure should be event-centred rather than only document-centred. Meaningful state changes leave events that can be replayed and interpreted. The exact low-level trace can be retained selectively for forensic needs, while scientific memory stores higher-level events such as hypothesis activated, experiment invalidated, dataset superseded, reviewer objection unresolved, or protocol approved. Such events are more useful to future agents than thousands of messages because they encode the transitions that changed what the project believed or permitted.

Continuity should not become total retention. Scientific work requires private thought, informal disagreement, and the freedom to make mistakes. Different classes of data need different retention policies. A meeting may preserve an accepted decision and a short rationale without keeping every personal remark. A model call may retain the inputs and outputs needed for reproducibility without storing hidden chain-of-thought. A project memory system should have explicit blind spots and be able to say when it cannot reconstruct a reason confidently.

The durable research project is therefore neither a database nor a narrative. It is a versioned, partially structured history of scientific objects, actions, evidence, and authority from which different views can be reconstructed. Documents are views. Dashboards are views. Briefings are views. AI context is a view. The project itself remains the persistent substrate.

This substrate changes the practical meaning of collaboration. A new researcher can join a project and ask what changed, which claims are disputed, what was abandoned, and why. A new model can be swapped into a role and evaluated against the same state. A partner can receive a bounded research object with provenance instead of a context-free file. A correction can propagate to dependent objects. These capabilities are mundane compared with the rhetoric of artificial discovery, but they are precisely what makes sustained machine participation scientifically credible.

Chapter 3 — Scientific Project State: The Substrate

Once the need for a persistent project object is accepted, the architecture requires a substrate precise enough to coordinate work without pretending to formalise science completely. Scientific Project State is that substrate. It is a deliberately small semantic layer around native artefacts: stable identities for questions, claims, evidence, experiments, decisions, constraints, and evaluations; versions and branches that preserve disagreement; and provenance that makes consequential transformations reconstructable. The purpose is practical rather than ontological. A project should be able to answer what changed, why it changed, what depends on it, and which parts of the record remain provisional. This chapter develops that substrate and tests the boundary between useful explicit structure and expensive knowledge engineering.

3.1 Objects, identity, evidence, and status

Scientific Project State is a small semantic layer that gives stable identity and status to the objects that determine scientific action. It should be rich enough to connect claims, evidence, experiments, evaluations, and decisions, but small enough to coexist with domain-specific data structures and ordinary repositories.

A research infrastructure needs a source of continuity, but it should resist the temptation to encode an entire discipline before useful work can begin. Grand knowledge-engineering projects often fail because the representation becomes more important than the research. The alternative proposed here is a minimal operational state: explicit only where identity, dependency, status, or authority affect what the project can do next.

The most fundamental object is the question. Questions change over time. A broad question may become several narrow hypotheses. A practical objective may be reformulated after a negative result. Keeping the question as an object prevents a common form of semantic drift in which agents successfully complete tasks that no longer answer the problem the team cares about. The question can therefore have versions, scope, relationships to subquestions, and links to decisions that changed it.

Claims are propositions whose scientific status matters. They can be candidate, supported, disputed, rejected, suspended, superseded, or domain-specific variants of these states. A claim is not stored as a truth value. Its status depends on evidence, evaluations, scope, and authority. Two incompatible claims can remain active simultaneously when the project has not resolved the disagreement. This preserves pluralism while making conflict visible.

Evidence links the claim to the world or to a source of constrained inference. It can refer to an observation, a dataset, an experiment output, a formal result, a source span, a reproduced computation, or another validated artefact. Evidence also has scope. One experiment may support a claim only under specific conditions. One paper may support the existence of an association but not a causal mechanism. The state should make it possible to represent this relation without pretending that every evidential judgment can be formalised.

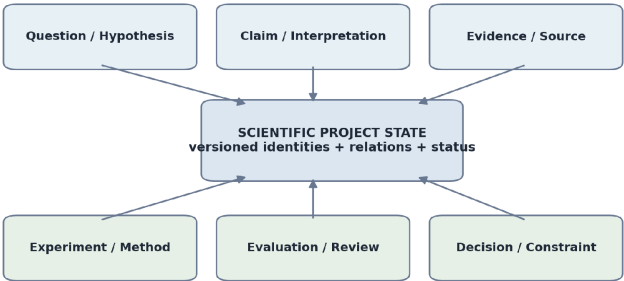
Experiments and methods describe how evidence was produced. The same conceptual experiment can have many runs with different versions, environments, instruments, or parameters. The project needs to distinguish the approved protocol from the actual execution and from the interpretation of the result. This distinction is already useful in computational work and becomes essential in physical laboratories.

Evaluations are first-class objects because a criticism should not overwrite the object being criticised. A claim can carry a successful citation check, a failed reproduction, a positive domain review, and an unresolved statistical concern at the same time. The project can then decide what status these assessments justify. An evaluator's identity, method, version, and declared scope travel with the evaluation so that later users know what passed and what remained unchecked.

Decisions record authoritative transitions. The project may decide to accept a claim provisionally, suspend a branch, change the main question, approve an experiment, or release an external artefact. A decision should identify the authority and the evidence considered. This record does not make the decision correct. It makes the responsibility and historical context recoverable.

Constraints capture boundaries that influence legitimate action. Some are scientific, such as a requirement that a benchmark remain uncontaminated. Some are operational, such as a compute budget. Some are legal or ethical, such as data-use restrictions. Some are safety-related. Constraints can be hard or advisory, and their source and version matter because they may change during the life of the project.

Scientific Project State: a small semantic core around real artefacts



Large files, code, datasets and raw logs remain in their native stores; the state layer preserves the scientific meaning that connects them.

Figure 3. Scientific Project State provides a small semantic core around native research artefacts.

The figure emphasises that Scientific Project State is not a replacement for native stores. Large datasets, code repositories, documents, raw instrument streams, and external publications remain where they are best managed. The state layer holds identities, versions, relations, and scientific status. This makes the architecture practical: the Research IDE can integrate a mature Git repository or laboratory information system instead of forcing all artefacts into a new proprietary format.

Table 8. A minimal scientific state needs semantic objects beyond files and conversational memory.

Object	Operational meaning
Question / hypothesis	What the project is trying to resolve and how the framing changes.
Claim / interpretation	A proposition whose scope, status, dependencies, and disagreement matter.
Evidence / source	An observation, computation, source, formal result, or validated artefact linked to a claim.
Experiment / method	The procedure, inputs, environment, expected observation, and actual execution episode.
Evaluation / review	A scoped assessment attached to an object without overwriting it.
Decision / constraint	An authoritative change in direction or a scientific, legal, ethical, resource, or safety boundary.
Artefact / task	The concrete document, dataset, code, figure, model, or operational work linked to scientific state.

The table provides a minimum vocabulary, not a universal ontology. A clinical project may add participants, cohorts,

endpoints, and consent objects. A mathematics project may add definitions, lemmas, conjectures, proof obligations, and formal artefacts. A software-security project may add threat models, vulnerabilities, attack traces, and mitigations. The common layer remains small enough that the platform can ask generic questions such as what supports this claim, which experiments depend on this dataset, or who accepted this transition.

Identity is more important than it first appears. Natural-language systems are tolerant of naming variation, but scientific provenance is not. If two agents refer to the same experiment with different descriptions, the project should still know whether they mean the same object. Stable identifiers allow prose to vary while dependencies remain exact. They also make citations within the project robust when titles change or documents are reorganised.

Status must be equally explicit. A generated interpretation should not enter persistent knowledge with the same status as a reviewed claim. A formally verified property should not imply that the empirical assumptions behind the formalisation are true. An unreproduced experiment should not become indistinguishable from a replicated result. The interface can hide complexity until needed, but the underlying project should retain the distinctions because they determine what actions are justified.

The state layer is therefore best understood as an epistemic coordination substrate. It does not decide truth. It gives the project enough structure to ask what would have to change for a claim's status to change, what evidence is missing, where disagreement resides, and which actions would be invalidated by a correction. Those questions become increasingly important as the number of automated participants grows.

The strongest empirical question for this layer is simple: does it improve research enough to justify the authoring and

maintenance cost? The answer should be measured on complete episodes. Can a team correct an error more locally? Can a new member reconstruct the project faster? Can a model be replaced without semantic reset? Can agent conflicts be detected before they corrupt artefacts? If a simpler combination of disciplined repositories and retrieval performs equally well, the state model should be simplified. The architecture is scientific only if it is willing to lose complexity when evidence does not justify it.

3.2 Versioning, branching, and semantic change

Research already branches intellectually even when its tools do not represent those branches. Scientific versioning should preserve alternative hypotheses, analyses, protocols, and interpretations, and it should distinguish a change in files from a change in meaning. A merge is therefore a scientific judgment, not merely a text operation.

Version control became foundational in software because many contributors cannot safely modify a complex artefact without recoverable history. Research has the same problem but with a larger object. A project version can include the current question, literature boundary, dataset, code, protocol, interpretation, review state, and decisions about what the team currently believes. Traditional file versioning captures fragments of this state while the scientific meaning remains informal.

AI increases the pressure because branches become cheap. One agent can test a different preprocessing method, another can use an alternative statistical assumption, a third can propose a rival explanation, and a fourth can rewrite the manuscript around its preferred interpretation. If these activities occur in one shared document, the project can converge accidentally on whichever agent writes last. Scientific branching gives alternatives durable identity so that disagreement survives long enough to be evaluated.

A branch should carry the assumptions that make it distinct. It may share data and methods with the main line while changing one interpretation. It may use a new dataset version. It may represent an exploratory high-risk direction that has not earned the same authority. The system can compare branches at the level of scientific objects rather than only file diffs. Researchers can ask which assumptions diverged, which evidence is shared, and what experiment could discriminate between the alternatives.

The idea of a semantic diff is crucial. Consider a manuscript sentence that changes from “the treatment improves recovery” to “the treatment is associated with faster recovery in one subgroup under the observed conditions.” The textual difference is modest. The scientific difference is large: causality became association, the population narrowed, and the claim acquired conditions. A project state should either detect or request this interpretation so that downstream objects can be reconsidered appropriately.

Semantic change appears in many forms. A concept can receive a new definition that invalidates comparisons. A simulation can change its boundary conditions. A literature review can discover prior art that removes a novelty claim while leaving the empirical result intact. A calibration update can invalidate some experiment runs but not others. A policy change can affect future data use without making historically authorised actions disappear. Generic file versioning cannot infer these distinctions reliably because it does not know what the objects mean.

A merge is therefore a scientific event. Two branches may be textually compatible and scientifically incompatible because they use different populations or standards of evidence. The environment should represent a merge proposal that identifies what can be combined, what remains disputed, what evidence favours each interpretation, and who has authority to select or

preserve alternatives. The result may be a synthesis, the acceptance of one branch, or the deliberate coexistence of both.

The familiar vocabulary of commits, branches, forks, and merges therefore acquires new meaning in research. The analogy to Git is useful precisely because it is incomplete. Research needs the recoverability of software versioning without pretending that intellectual conflicts can be solved by line-level merge algorithms.

Forks also support independent evaluation. A reproduction team can work from a controlled snapshot without modifying the original project. A regulator can inspect a bounded copy under a different policy. A company can expose one part of a proprietary project to a partner while retaining internal branches. The differences among forks become evidence about transfer: which assumptions, methods, or environments remain valid outside the original context.

Version graphs may eventually become valuable training resources. A final paper teaches a model the language of accepted results. A project history teaches how claims were narrowed, how attractive ideas failed, how reviewers changed methods, and how uncertainty was resolved or preserved. These traces could support research agents that learn scientific correction rather than merely imitate scientific prose. The privacy and ownership risks are substantial, so permission to assist a project must remain separate from permission to train on its history.

Versioning also enables temporal integrity. Historical benchmarks should use the literature and project memory available at the target date. A system that has already seen the future answer should not be credited with independent discovery. The same principle applies to long-term agent memory. A research environment needs explicit memory scopes so that procedural knowledge, project history, and external

corpora do not contaminate evaluations intended to measure genuine transfer.

The limit of versioning should remain visible. A complete history does not establish truth. A signed commit can record a bad decision. A reproducible computation can answer the wrong question. A well-preserved branch can contain a shared misconception. Version control provides continuity; evaluation provides challenge; empirical evidence provides contact with the world; community acceptance provides social authority. The architecture becomes safer when these layers do not borrow one another's names.

This separation also keeps the interface honest. A dashboard should not show one green badge labelled verified when the underlying state is more complex. It can show that provenance is complete, reproduction succeeded, a statistical concern remains open, and no external replication exists. The richer state may be less comforting, but it is more useful for scientific judgment.

3.3 Provenance across tools and institutions

Provenance connects scientific outputs to sources, transformations, environments, actors, rights, and decisions. It should be captured during work rather than reconstructed at publication, and it must remain usable when projects cross organisational and technological boundaries.

Within one repository, version control can provide strong history. Research rarely remains inside one repository. A dataset may come from a hospital, an algorithm from a university, a model from a vendor, an analysis from a company, and a physical experiment from a contract laboratory. Each organisation has different identities, access controls, retention rules, and evidence systems. The scientific result depends on all of them, and the project needs a way to preserve that dependence without centralising everything.

Provenance answers a narrow but powerful set of questions. Where did this artefact come from? Which activity transformed it? Which people, tools, models, or institutions participated? Which version of the input was used? Under which permissions did the action occur? Which decision accepted the result? These questions do not establish whether the result is scientifically correct. They establish whether the path to the result can be reconstructed and challenged.

The right time to capture provenance is when the action occurs. A data transformation already knows its inputs and method. A model call already knows the model endpoint and project context supplied to it. A container run already knows the code revision and environment. An approval already knows the authority and the object being approved. Asking authors to reconstruct this information months later guarantees gaps and encourages retrospective simplification.

The infrastructure should therefore make evidence capture a runtime property. Consequential tool calls leave receipts. Derived artefacts link to their inputs. Experiment runs identify their protocol and environment. Model-generated contributions identify the role and version used. Human approvals reference the evidence reviewed. The project can later generate a concise disclosure or audit view from the same underlying record without forcing every reader to inspect operational detail.

Cross-institution sharing requires selectivity. A partner may need to verify that a result was produced from an approved dataset without receiving the dataset itself. A hospital may expose a remote reproduction capability rather than export patient data. A company may provide a signed evaluation while keeping proprietary code private. The shared project can therefore exchange bounded research objects that include stable identity, version references, declared evidence, rights metadata, and receipts while sensitive content remains local.

OpenDSU is relevant to this direction because it explores identity, controlled sharing, and version continuity close to data objects. It should not be treated as the mandatory implementation. W3C PROV, RO-Crate, content-addressed storage, signed attestations, conventional access-control systems, and domain-specific infrastructures can all contribute. The research question is which minimal provenance model produces operational value without overwhelming projects with metadata.

This perspective demonstrates that provenance is useful for correction, not merely audit. If a dataset is later found to contain an error, the system can identify which analyses, figures, claims, and decisions depend on it. If an external tool is compromised, affected runs can be quarantined. If a consent condition changes, future actions can be blocked while the historical record of earlier authorised use remains intact. Dependency-aware remediation turns provenance into a scientific resilience mechanism.

Provenance also improves attribution in human-AI work. A generic sentence that AI was used becomes inadequate when models participate throughout a project. The infrastructure can record that one agent found sources, another generated code, a solver checked a property, a human revised the interpretation, and an editor approved publication. Formal authorship can remain human while contribution becomes precise. This protects both accountability and credit.

The record should not become surveillance. Research teams need spaces for speculative conversation and private reasoning. Not every token, keystroke, or failed thought should be permanent. The important distinction is between operational trace and scientific memory. Low-level logs can have short retention or restricted access. Persistent provenance should concentrate on actions and transformations that affect evidence, rights, resources, or authoritative state.

Interoperability should also occur at the level of meaning rather than vendor-specific agent messages. A partner should be able to provide an evidence object or evaluation receipt without adopting the same vector database, LLM provider, or workflow engine. Typed scientific objects create a narrower and more durable contract. They let institutions combine heterogeneous systems while keeping their internal technology choices independent.

A provenance layer that survives organisational boundaries is therefore a prerequisite for collaborative research at machine speed. It allows the project to know that two partners are referring to the same object, that one evaluation was performed under a particular method, and that a later correction affects specific downstream claims. The architecture becomes federated without becoming semantically fragmented.

The strongest test is practical. When something goes wrong, can the project identify the affected scientific objects quickly enough to repair them? When a new collaborator joins, can the system explain where an important result came from without exposing material they are not authorised to see? When a model is replaced, can the organisation distinguish changes caused by the model from changes caused by data or method versions? Provenance earns its place when it shortens these real correction and coordination tasks.

Chapter 4 — From Scientific Intent to Executable Work

State alone does not perform research. The difficult step is turning an evolving scientific intention into machine-assisted work without allowing the operational interpretation to silently redefine the question. This chapter follows that translation. It treats the specification as a living scientific constitution, not as a bureaucratic plan; executable work as progressively formalised rather than universally formal; and orchestration as coordination under budgets, permissions, stopping conditions, and revision. The objective is to make semantic drift locatable. A successful execution must not automatically become a successful scientific claim, and an agent must not acquire authority merely because it can perform an operation. The infrastructure becomes useful when it preserves these distinctions while still allowing rapid exploration.

4.1 Living specifications and scientific purpose

When implementation becomes cheap, weakly specified intent becomes expensive. A living research specification preserves the question, values, assumptions, evidence expectations, authority boundaries, and unresolved ambiguity that should guide execution. It is a scientific constitution, not a prediction of the answer.

Generative AI changes the economics of implementation before it changes the economics of intent. Code, analyses, diagrams, reports, and experiment plans can be produced rapidly. If the purpose is poorly defined, the system can produce an entire forest of plausible work that is internally coherent and scientifically irrelevant. The faster the execution layer becomes, the more important it is to know what the project is actually trying to accomplish.

A conventional software specification often describes required behaviour in advance. Research cannot always do that because discovery changes the problem. A useful scientific specification is therefore living and explicitly incomplete. It describes the current question, the values that matter, the assumptions being tolerated, the kinds of evidence that would count, the constraints that must not be violated, and the authority required to change direction. It also records ambiguity instead of filling every gap with confident prose.

This is why the metaphor of a constitution is more useful than the metaphor of a contract. A constitution does not decide every future case. It defines legitimate roles, boundaries, procedures, and methods for revising itself. A research specification should orient action while allowing the project to learn that its original framing was wrong. The specification is successful when a later change can be explained, not when the original document remains untouched.

Tao's argument about the multiple purposes of mathematics shows why one objective function is insufficient. Scientific work can value correctness, understanding, explanatory power, novelty, reproducibility, safety, learning, community infrastructure, or practical utility. These values often correlate, which is why visible outputs such as papers or benchmark improvements can serve as rough proxies. AI can exploit the gap between the proxy and the wider purpose. A system told to maximise novelty can generate unusual but meaningless combinations. A system told to improve a benchmark can overfit. A system told to automate review can optimise the appearance of rigorous criticism rather than help experts find errors.

The specification should therefore preserve several values without collapsing them prematurely. Early exploration may value information gain and breadth. A later publication stage may require reproducibility and clear explanation. A clinical

project may impose safety and privacy constraints that dominate convenience. A training project may deliberately sacrifice speed so that junior researchers develop judgment. The infrastructure should know which stage and values apply to the current transaction.

Executable research preserves intent by making interpretation and revision visible

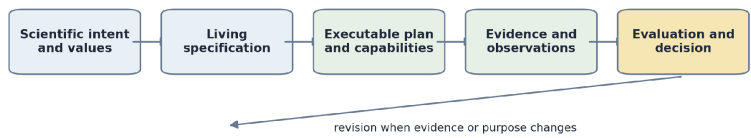


Figure 4. Specification-driven research keeps purpose connected to execution and revision.

The figure places revision inside the loop rather than treating it as failure. Scientific intent becomes a living specification; the specification guides plans and capabilities; execution produces evidence; evaluation may change the decision or the question; and that change can legitimately revise the specification. The important requirement is that the revision itself remains visible and attributable. Execution should not silently redefine the research question simply because a tool found a convenient interpretation.

Table 9. The living specification keeps purpose connected to execution and revision.

Layer	What the project must preserve
Scientific purpose	The question, wider values, why the work matters, and what should not be reduced to one metric.
Living specification	Assumptions, alternatives, evidence expectations, authority boundaries, stopping conditions, and unresolved ambiguity.

Layer	What the project must preserve
Executable plan	Tasks, people, agents, tools, data, dependencies, budgets, permissions, and reversible branches.
Evidence record	What was actually executed, which versions were used, what was observed, what failed, and what remains uncertain.
Evaluation and decision	Independent checks, explanation, disagreement, repair, rejection, approval, or provisional retention.
Versioned history	How intent and interpretation changed and which earlier decisions may need reconsideration.

The table separates purpose from plan. This prevents a common form of agentic drift. A plan can fail operationally while the purpose remains valid. A plan can succeed while the purpose remains unmet. A new result can justify changing the purpose itself. When these layers are represented separately, the project can locate where misalignment occurred instead of narrating the final outcome as if every step had been inevitable.

A specification also needs prohibitions and unresolved questions. Some actions are not allowed even if they improve the local objective. A model may not expose confidential material to an external service. An experiment may not vary a parameter that is fixed by scientific design. A novelty assessment may not treat lack of retrieval as proof that no prior art exists. A statistical analysis may not change the primary endpoint after seeing the result without recording that change. These boundaries become executable where possible and remain explicit human constraints where formalisation is inadequate.

The specification should state what a measure is evidence for and what it is not allowed to decide. An accuracy score can compare models on a benchmark; it does not define scientific significance.

A successful laboratory run establishes that an operation occurred under recorded conditions; it does not prove the mechanism proposed in the introduction. A citation count can indicate attention; it does not establish correctness. This simple discipline is one defence against Goodhart-style drift.

Natural language remains essential. The point is not to replace scientists with a domain-specific language. The point is to record the operational interpretation used for consequential actions. A researcher can write an intent in ordinary language. The system can propose a structured reading, expose ambiguities, and bind the chosen interpretation to the resulting plan. When another reading is plausible, the project can preserve both or request clarification. Language remains expressive while execution becomes inspectable.

The strongest test of a specification is whether it makes misalignment easier to see. Can a reviewer identify that an agent optimised a proxy? Can the team see which assumption entered during planning but was never justified by the scientific purpose? Can the system refuse to promote a technically successful result because the required evidence or explanation never appeared? Can researchers revise the objective without losing the history of why the change occurred? If the answer is yes, the specification is serving science rather than merely organising tasks.

4.2 Progressive formalisation and executable capabilities

Executable science does not mean translating every scientific idea into code. It means increasing precision where repetition, risk, coordination, or verification justify it. Natural language, diagrams, procedures, programs, formal constraints, and human judgment can coexist, with stable fragments gradually becoming qualified reusable capabilities.

Research often begins where vocabulary is unstable. A team may not yet know which variable matters, whether two observations belong to the same mechanism, or which distinction deserves formal expression. Forcing complete formalisation at this stage can turn a provisional interpretation into infrastructure and make the project efficiently wrong. The credible alternative is progressive formalisation.

The project begins with a natural-language question and a few explicit boundaries. As work repeats, stable fragments become more structured. A literature search that initially depends on an expert's ad hoc queries may acquire a documented corpus, inclusion logic, and reproducible search receipts. A data transformation that was once improvised by a coding agent may become a tested procedure. A compliance interpretation may be divided into a deterministic rule for unambiguous cases and a human escalation path for contested cases. A laboratory protocol may gradually acquire machine-enforceable preconditions and safety limits.

Different parts of the same project can therefore have different levels of executability. A high-level research question may remain interpretive. A data pipeline can be deterministic. A model-generated plan can be provisional. A formal solver can verify a narrow property. A human review can determine whether the formalisation captures the intended claim. The infrastructure should preserve these boundaries rather than advertise the entire chain as formally verified because one step has a strong guarantee.

This is where neuro-symbolic and explicit process representations become useful. Neural models are good at reading ambiguous language, proposing analogies, and generating candidate structures. Symbolic or conventional programs are better at exact identity, dependency, deterministic transformation, policy enforcement, and reproducible execution.

Formal solvers can establish properties inside explicit assumptions. Humans remain necessary for purpose, theory choice, contested meaning, and social authority. Hybrid architecture is justified by the heterogeneity of scientific work rather than by a desire to combine fashionable paradigms.

Axiologic's MRP-VM and SOP Lang circuit experiments belong to this middle layer. Their value is not that one notation should become a universal language of science. Their research question is whether recurrent language-mediated work can be decomposed into explicit operations, dependencies, alternatives, and validations so that parts of the process become inspectable and replaceable. If a simpler workflow engine or ordinary program solves the same problem more reliably, the simpler mechanism should be used.

The boundary of every guarantee matters. A solver verifies the formal statement it receives; it does not prove that the statement faithfully represents the original scientific question. A statistical library can compute a correct estimate while the study design remains inappropriate. A model can summarise a paper accurately while the paper itself is wrong. Formalisation increases trust only when the translation into the formal object remains visible.

Progressive formalisation is best understood as a ladder of responsibility rather than a hierarchy of intellectual prestige. More formal is not always better. A representation is valuable when it provides leverage appropriate to the task. The goal is to formalise the smallest stable fragment that improves action, verification, or reproducibility and to leave the rest flexible until evidence justifies more structure.

This approach leads naturally to executable capabilities. A successful agent response is situated: one model, one context, one set of hidden assumptions. A capability makes a stronger claim. It says that a recognisable responsibility can be performed across a

defined family of cases with known inputs, outputs, tools, permissions, failure conditions, and evaluations. The organisation should not promote one lucky interaction into a reusable capability without testing transfer and boundary behaviour.

Qualification therefore requires negative cases. A citation checker must not only find obvious mismatches but abstain when the source is inaccessible or the claim is too transformed. A laboratory planner must know that a procedure valid for one instrument may be unsafe for another. A review agent must not repeat the same fashionable criticism whenever it sees certain keywords. Qualification tests whether the capability knows its scope, not merely whether it succeeds in favourable examples.

Capabilities should be versioned and replaceable. A new model may improve average performance while creating a dangerous regression in one domain. The system should be able to compare the candidate implementation with the active one, inspect cost and failure changes, and restore the earlier version. Adaptation without qualification is merely drift with optimistic language.

Progressive formalisation also creates institutional learning. The organisation does not accumulate only chat transcripts. It converts repeated successful patterns into explicit assets and repeated failures into explicit warnings. Over time, some tasks require less open-ended generation because the project has learned a reliable procedure. Other tasks remain interpretive because their meaning genuinely changes. The architecture can represent that boundary instead of forcing all work toward either full autonomy or permanent manual control.

4.3 Orchestration, budgets, and controlled execution

Research orchestration coordinates heterogeneous capabilities around a changing scientific state. It should manage branches, dependencies, resources, stopping conditions, evaluation requests, and escalation without pretending to know the scientific answer in advance. Execution must be sandboxed, observable, and separable from authority.

The word orchestration often evokes a workflow diagram in which every step is known before execution. Research rarely behaves that cleanly. A literature result can change the hypothesis. A failed reproduction can create a new question. A statistical concern can require a different experiment. The orchestrator therefore needs to support both fixed workflows and exploratory planning while preserving the state transitions that connect them.

A simple task may use an explicit recipe. Retrieve a paper, locate the cited passage, compare it with a claim, and return an evaluation object. A more open task may require a planning model to propose several branches, estimate their cost, choose one under a budget, and revise after evidence arrives. The infrastructure should permit several orchestration strategies under the same contract so that they can be compared experimentally. A complicated hierarchy of agents should not survive merely because it looks sophisticated if a small library of explicit procedures performs equally well.

Budgets are part of scientific control. Agentic systems can waste compute by exploring low-value variations, retrying broken environments, or launching expensive searches whose expected information value is small. Existing schedulers optimise throughput or job completion. Research orchestration must also expose why resources are being spent. A branch should have a

budget, a stopping condition, and a record of what the spend produced. The system does not need a perfect theory of scientific value to make waste visible.

Stopping is as important as starting. An agent should not continue searching simply because another token or experiment is available. A literature search can stop when coverage is adequate for the current decision. A reproduction attempt can stop after the missing dependency makes further retries uninformative. An exploratory branch can be suspended when its expected learning value falls below a configured threshold. The decision to stop can be revisited later if constraints change.

Controlled execution means that technical capability does not imply permission. Code should run in a sandbox with explicit data mounts, network rules, secrets, and resource limits. Retrieved content should be treated as untrusted because papers, repositories, and web pages can contain prompt-injection text or malicious code. A model that reads such content should not automatically receive authority to send external messages or access confidential data. Security boundaries belong below natural-language instructions.

Every meaningful run should produce a receipt. The receipt does not need to expose internal model reasoning. It records the role, implementation version, project state snapshot, inputs, tools, environment, outputs, resource use, and consequential actions. This is sufficient for reproducibility, attribution, and incident response. The runtime can retain more detailed logs under appropriate retention policies when forensic analysis is needed.

This architecture positions orchestration as coordination rather than judgment. The planner can decide how to attempt a task within the specification, but evaluation determines what was established, and authority determines what consequences follow. This separation makes agent failures easier to localise. A bad plan is different from a correct plan executed against the wrong

dataset. A good execution is different from a scientifically irrelevant result. A valid result is different from an authorised publication.

Concurrency makes the separation particularly valuable. Several agents can work in parallel on different branches. Each reads a state version and returns candidate changes. If two candidates conflict, the project can preserve both, request reconciliation, or apply a domain-specific rule. The first agent to finish does not win by accident. This is the scientific analogue of optimistic concurrency control, with the important difference that some conflicts are legitimate intellectual disagreements rather than technical errors.

The orchestrator should use evaluation feedback without allowing evaluation to become self-confirmation. A branch with repeated reproduction failures can be deprioritised. A contradictory result may receive additional replication budget. A novelty search may redirect work away from a known method. But the planner that generated the branch should not be the sole judge of whether its own evidence is sufficient. Heterogeneous evaluation protects against a single model turning its preferences into project policy.

Research orchestration is therefore an empirical field in its own right. Does dynamic planning improve scientific yield compared with simple workflows and human scheduling? Does value-of-information estimation reduce expensive experiments, or does it introduce brittle assumptions? Do specialised agents outperform a capable general model once coordination overhead is counted? The infrastructure should make these strategies swappable so that architectural choices become testable hypotheses rather than permanent commitments.

The objective is not to create the appearance of an autonomous organisation. It is to make machine labour legible enough that humans can use much more of it without losing control over

purpose, evidence, and consequence. A strong orchestrator is quiet when the work is routine, explicit when branches diverge, and willing to stop when additional automation would generate more activity than knowledge.

Chapter 5 — Agents as Bounded Scientific Contributors

Machine participation becomes governable only when the word agent stops functioning as a synonym for an artificial colleague. The more stable unit is a scientific role with declared inputs, project view, tools, evidence requirements, failure behaviour, and authority. This chapter develops that division of labour. It explains why capability and authority must be separate, how different implementations can occupy the same role, and how recurrent improvisation can become a qualified reusable capability without turning a successful prompt into an institutional rule. The aim is not to maximise the number of agents. It is to create a research organisation in which machine labour can expand while responsibility, replacement, and failure localisation remain intelligible.

5.1 Roles rather than personalities

The useful unit of design is a scientific role, not the fictional image of an artificial colleague. Agents can search, transform, test, critique, coordinate, monitor, or explain, but each role should have a bounded responsibility and should remain subordinate to explicit project authority.

The word agent is attractive because it makes software feel social. A model is given a name, a job description, and a persona, then invited to behave like a digital colleague. This can improve interaction, but it is a weak basis for infrastructure. A personality does not tell the platform which data the agent may read, which tools it may invoke, which objects it can change, how its output

should be checked, or what should happen when it encounters a case outside its competence.

Research environments need a less anthropomorphic definition. A literature role searches declared sources and returns candidate evidence with search scope and provenance. A reproduction role rebuilds a computational environment and reports what could and could not be reproduced. A statistical role checks a declared class of analyses. A planning role proposes next actions under a budget. A compliance role evaluates an intended action against configured normative sources. A laboratory role translates an approved protocol into instrument operations within a safety envelope. Project Consciousness observes authorised history without acquiring scientific authority.

The distinction matters because the role is more stable than the implementation. The literature role can be performed today by a language model with retrieval tools, tomorrow by a stronger model, and later by a specialised combination of search, symbolic query expansion, and deterministic filters. The user should continue to receive the same kind of evidence object and the project should continue to enforce the same authority boundary. A research organisation can then improve machinery without redefining its scientific constitution every time a vendor releases a new model.

Role-based design also localises failure. Saying that the agent was wrong is not enough. Did retrieval fail to cover the relevant corpus? Did planning introduce an unsupported assumption? Did the execution layer use the wrong file version? Did a statistical evaluator apply a test outside its domain? Did a human approve a result despite unresolved evidence? The ability to name the failed responsibility is a prerequisite for improving the system.

Tao's insistence on human authorship and responsibility is relevant here. Scientific authorship is not only a record of who generated text. It is a commitment to understand, defend, correct,

and accept consequences for the work. Current AI systems cannot bear that institutional responsibility. A model should therefore not become the moral owner of a result merely because it produced a large fraction of the code or analysis.

This does not mean machine contribution should be hidden. A generic disclosure that AI was used becomes almost meaningless when machine participation is pervasive. The infrastructure can record that one model proposed a hypothesis, another generated code, a solver checked a formal property, a robotic system executed a protocol, and human researchers interpreted and accepted the result. Authorship remains a human and institutional convention while contribution becomes operationally precise.

This distinction demonstrates why specialisation is about responsibility rather than theatrical multiplicity. A project does not need twenty agents merely to create a sense of sophistication. It needs roles that have different evidence requirements or authority boundaries. Some roles may use the same model under different harnesses. Others may be deterministic tools that barely resemble agents. The platform should care about what the role promises and what the project can verify.

A RoboTeam, in this sense, is a configurable organisation of bounded contributors. During one research meeting, the team may include a literature role, a modelling role, a critic, and a documentation role. During a reproduction task, a different set becomes active. Roles can remain dormant until the state makes them relevant. This avoids the overhead of a permanent society of models talking to one another simply because the architecture is labelled multi-agent.

The shared project state replaces the fiction that agents must share one mind. Each role receives an authorised project view: the subset of questions, claims, artefacts, versions, and permissions needed for its task. A compliance role may need

metadata about a protected dataset without access to its contents. A literature role may need the scientific question but not confidential partner data. A critic may need the proposed claim and evidence while remaining intentionally isolated from the planner's rationale to reduce correlated bias.

This view-based design improves both privacy and scientific interpretation. When two agents disagree, the environment can ask whether the difference came from model behaviour, evidence access, or assumptions. An output can be judged relative to what the role actually saw. Narrower context can also improve model performance by reducing irrelevant material and decreasing opportunities for accidental leakage.

The user interface can still present coherent collaboration. Researchers may prefer to talk to one general copilot that delegates behind the scenes. The important requirement is that the project record distinguishes the responsibilities that were invoked. A conversational surface should not collapse retrieval, computation, verification, and authority into one voice simply because one chat window is convenient.

Role-based agents therefore support a division of scientific labour without pretending that the system has created artificial persons. The design question becomes practical: what responsibility is being delegated, what evidence accompanies the delegation, and what consequence is the role allowed to create? This is a much stronger foundation for scientific trust than a name, a persona, or a long prompt describing how careful an agent should be.

5.2 Capability, project view, authority, and evidence

Every agent role can be described through four operational dimensions: capability, project view, authority, and evidence.

Qualification adds a fifth dimension by establishing whether a particular implementation deserves to occupy the role within a declared domain and risk envelope.

Capability answers what the role can do. A useful capability is narrow enough to test but broad enough to reuse. It declares the input object types it accepts, the output it produces, the tools it may use, the situations in which it should abstain, and the resources it may require. The declaration does not need to capture every internal step. It provides a stable contract between the project and a changing implementation.

Project view answers what the role is allowed to see. Context should be assembled from project state rather than inherited wholesale from a user's account. The view identifies relevant claims, artefacts, versions, data classifications, and permissions. It can also intentionally exclude information. An independent evaluator may be more useful if it does not see the generator's preferred conclusion. A historical benchmark agent may be prohibited from accessing literature published after the target date.

Authority answers what the role may change or cause. This dimension must be independent of capability. A model may be technically capable of submitting a paper, transferring data, or controlling equipment through available tools, yet the project should not grant that authority simply because the action is possible. Authority can be scoped by object type, action, cost, reversibility, location, and required approvals.

Evidence answers what the role must reveal about its work. A retrieval result should identify sources and search scope. A computation should identify code, data, environment, and parameters. A recommendation should expose the assumptions and alternatives it considered at the level necessary for review. An external action should leave a receipt. Evidence turns a transient answer into an inspectable project event.

A scientific agent is a governed capability, not a personality

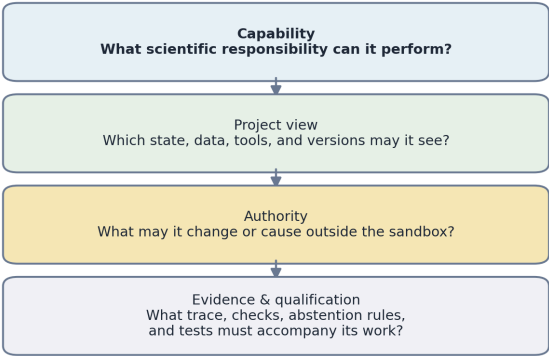


Figure 5. A scientific agent is a governed capability, not a personality.

The diagram places these dimensions in a simple stack because they constrain one another. A role can have a strong capability and still receive a narrow view. It can see broad context and still have almost no authority. It can be allowed to produce a candidate while being required to attach strong evidence before the candidate reaches review. This separation makes it possible to exploit model capability without giving the model an equivalent level of trust.

Table 10. The operational contract of a scientific agent role.

Dimension	Meaning in the research environment
Capability	The bounded responsibility, accepted inputs, produced outputs, tools, abstention conditions, and resource expectations.
Project view	The authorised subset of state, data, versions, and context supplied to the role.
Authority	The changes or external consequences the role is permitted to create; granted separately from technical capability.

Dimension	Meaning in the research environment
Evidence	The sources, execution receipts, assumptions, checks, and provenance that must accompany the role output.
Qualification	Evidence that one implementation performs the role reliably inside a declared domain and risk envelope.

The table makes the contract implementable. An agent manifest can declare accepted input types, output types, required tools, data sensitivity limits, expected cost class, and required evaluators. A tool adapter can declare its parameters and side effects. A policy can state that a role may create draft objects but not accepted claims. These contracts can remain simple in early prototypes and become richer only where operational evidence demands it.

Abstention is an important part of the contract. An agent that cannot access key prior art should not declare novelty. A reproduction role that cannot rebuild the environment should distinguish infrastructure failure from scientific non-reproduction. A compliance role that cannot determine data classification should escalate rather than guess. Reliable abstention is often more valuable than a small increase in average answer rate in high-consequence research.

The authority model also supports graduated autonomy. A new capability can begin in suggestion mode, where it only creates candidate objects. After successful qualification, it may gain authority to run reversible sandboxed work automatically. Later, a mature role may commit low-risk state changes under policy while consequential decisions still require human approval. If regressions appear, authority can be reduced without deleting the capability. Autonomy becomes an evidence-based operating policy rather than a philosophical label applied to the whole system.

Human roles should be represented with similar clarity. A principal investigator, statistician, data steward, laboratory manager, external reviewer, and editor do not have interchangeable authority. The infrastructure can make these differences explicit without forcing users to think in access-control jargon. A familiar interface can show that an experiment plan awaits scientific approval, a data transfer awaits a steward, and a publication decision belongs to an editor. The underlying system enforces the boundaries consistently for humans and machines.

Evidence requirements can also vary by consequence. A quick brainstorming role may return prose with lightweight provenance. A result intended to become part of accepted project knowledge may require source links, execution receipts, and independent checks. A physical action may require an approved protocol, safety preconditions, and a lower-level interlock. The project does not need maximum traceability for every thought; it needs proportional traceability where consequences justify it.

This contract model is one of the most practical ways to turn AI from a conversational feature into research infrastructure. It enables model replacement, controlled federation, independent testing, and clearer incident response. If a role fails, the organisation can inspect whether the problem lies in the implementation, the project view, the authority boundary, or the evidence requirement. Improvement becomes a systems engineering task rather than an exercise in rewriting personalities.

5.3 Qualification, progressive stabilisation, and model replacement

Successful AI improvisation should not silently become institutional procedure. Recurrent work should be qualified through transfer tests, negative cases, evidence requirements, and versioned promotion. Over

time, stable parts of the work can become more deterministic while open-ended interpretation remains generative.

AI agents are valuable because they can improvise. They can read an unfamiliar repository, interpret an unusual document, combine tools, or propose a procedure that was not predefined. Research needs this flexibility because novel problems are not fully specifiable in advance. The danger begins when one successful improvisation is copied into long-term automation without establishing where it works and where it fails.

A response belongs to one context. A qualified capability makes a stronger institutional claim. The path from one to the other should be explicit. The team first identifies the reusable responsibility rather than saving the whole transcript. It defines the expected inputs and outputs, the tools and permissions required, the situations in which the capability should abstain, and the evidence that must accompany the result. It then tests the implementation across varied cases, including cases designed to expose over-generalisation.

Negative tests are especially important for generative systems. A literature agent that finds relevant prior art in familiar terminology may fail when the field uses different vocabulary. A statistical critic may learn to produce standard warnings that sound rigorous but do not apply to the specific design. A planning agent may produce safe protocols for one instrument family and dangerous ones for another. Qualification should probe these boundaries rather than reward only successful demonstrations.

The qualification record should be versioned. A model update, prompt change, tool upgrade, retrieval corpus, or runtime modification can affect behaviour. The active implementation should be compared with a candidate on representative tasks and important failure cases. Cost and latency also matter because a capability that is slightly more accurate but ten times more expensive may not improve the complete research process.

This leads to progressive stabilisation. Some tasks begin as open-ended model work and become more structured as the organisation learns. A document transformation that required interpretation may eventually become a conventional program. A policy check may split into deterministic rules for clear cases and a generative service for ambiguous interpretation. An experiment preparation procedure may become a tested workflow with only one remaining step that requires model judgment.

Progressive stabilisation is not a march toward symbolism for its own sake. Some scientific tasks will remain interpretive because the concepts are contested or the evidence changes. The objective is to give stronger guarantees to recurrent, high-consequence fragments when the cost of doing so is justified. A model remains useful at the frontier; stable infrastructure accumulates behind it.

Qualification should measure the complete role rather than a single benchmark score. Reliability includes failure behaviour, provenance quality, abstention, human correction effort, cost, privacy, and transfer. A role that produces fewer answers but reliably identifies when it lacks evidence may be more valuable than one that maximises completion rate while hiding uncertainty.

Model replacement then becomes an ordinary lifecycle event. The project role remains stable, and a new implementation enters with limited authority. It can be evaluated on current and historical cases, perhaps running in shadow mode beside the incumbent. If it performs better within the risk envelope, the organisation can promote it. If a later regression appears, an earlier version can be restored. The project history remains intact because scientific state is not stored inside the model.

This lifecycle also helps institutions avoid hidden procedural memory. Many agent systems accumulate tricks in prompts,

examples, and private context until the behaviour is difficult to reproduce. A qualified capability should expose the parts of that memory that materially affect its role. Reusable methods, scripts, prompts, and model configurations become versioned assets with declared scope. They can be retired when evidence changes.

The research value of MRP-VM, SOP circuits, agent libraries, and controlled deployment can be judged through this lens. Do they help convert useful improvisation into inspectable competence? Can the same role be implemented by a different model without rewriting the project? Can failures be localised? Can stable procedures be replayed without asking a model to rediscover them each time? These questions are stronger than whether an agent appears intelligent in a demonstration.

A further benefit of qualification is comparative procurement. Research organisations should not have to choose a model brand and then discover later which scientific roles it can safely perform. They can instead define the role first and invite several implementations to compete against the same evidence and failure cases. This changes the purchasing question from which model is smartest to which implementation is most reliable, economical, private, and correctable for this responsibility. It also reduces strategic dependence on one provider because replacement is planned into the contract rather than treated as an emergency migration.

Qualification also creates a natural boundary for sharing capabilities between institutions. A partner can expose a role with declared inputs, outputs, cost, privacy conditions, and evidence without revealing the proprietary mechanism behind it. The receiving project can decide whether the qualification evidence is sufficient for its own risk level and can require local reevaluation when necessary. This is more realistic than expecting every consortium partner to use the same agent framework or model stack.

The ultimate measure is organisational learning. A research environment should become easier to trust after sustained use because it has accumulated qualified capabilities, known failure modes, tested evaluators, and explicit authority policies. If the system simply accumulates longer chat histories and more opaque prompts, it has gained memory without gaining infrastructure. Progressive stabilisation turns experience into durable scientific capacity.

Chapter 6 — Evaluation, Review, and Scientific Digestion

The architecture matters most after generation. A result can be fluent, executable, reproducible, or formally checked and still fail to deserve scientific authority. This chapter therefore moves from production to evaluation and digestion. It treats evaluation as a portfolio of bounded guarantees rather than a single confidence score, and it treats review as an allocation of scarce expert attention rather than an automated verdict. The same distinction extends beyond publication: explanation, replication, disagreement, use, and canonicalisation are separate stages through which candidate output becomes shared knowledge. The infrastructure should make those stages visible, help experts challenge consequential claims earlier, and preserve uncertainty when no evaluator has earned the right to collapse it.

6.1 Evaluation as a portfolio of independent checks

Verification is not one operation and should not produce one universal green badge. Scientific evaluation is a portfolio of checks with different scopes and failure modes: source support, reproducibility, statistical or methodological validity, formal properties, contradiction, novelty, explanation, and human review. Their results should coexist without being collapsed into a single model score.

A future research environment will generate more candidate results than experts can inspect manually. The obvious response is to automate evaluation, but that phrase hides an important danger. If the same system generates a claim, evaluates the claim,

and then summarises the evaluation as confidence, the project has created self-confirmation rather than independent evidence. The infrastructure should therefore treat evaluation as a separate class of scientific work.

The first kind of evaluation concerns source support. A citation can be checked for existence, accessibility, and whether the cited passage actually supports the attributed claim. This is a bounded task in which retrieval and language models can help substantially. The output should distinguish direct support, partial support, contextual relevance, contradiction, and inability to verify. A polished citation string without source-level evidence should not count as successful checking.

Reproducibility is a different question. A computational result may require code revision, data identifiers, dependency versions, parameters, random seeds, hardware-relevant metadata, and execution logs. A reproduction role attempts to rebuild the environment and rerun the procedure. Failure can mean that the package is incomplete, that the environment no longer exists, that the numerical result differs, or that the scientific conclusion changes. These outcomes must not be compressed into one reproduced or not reproduced label.

Methodological evaluation asks whether a reproducible procedure was scientifically appropriate. Statistical checks can inspect train-test leakage, missing controls, unstable estimates, inappropriate causal language, or deviations from a declared protocol. Formal tools can verify properties in domains where formalisation is faithful. Domain-specific validators can detect impossible physical or logical states. A language model can help interpret these checks but should not inherit the guarantee of the deterministic tool it invokes.

Contradiction is another independent signal. Evidence can fail to support a claim without actively supporting the opposite. A contradiction search looks for incompatible results in the project

or external literature. The correct response to contradiction is often a new question rather than immediate rejection: different populations, definitions, environments, or assumptions may explain the disagreement. The state should therefore support competing claims instead of forcing truth values prematurely.

Novelty deserves particular caution because it is partly a property of search coverage. A system can find no prior art and still be wrong because it searched the wrong vocabulary or corpus. A useful novelty service decomposes the contribution, constructs several retrieval strategies, identifies the strongest antecedents, and reports which aspects appear known or different. It should say potentially novel within documented coverage rather than convert absence of retrieval into certainty.

Table 11. Evaluation is a portfolio of bounded guarantees rather than one score.

Evaluation	What it can establish	What it does not establish by itself
Source support	Whether identified source material supports, qualifies, or contradicts a claim.	That the source is true or the claim is important.
Reproduction	Whether a declared procedure can be rebuilt and rerun with comparable output.	That the method was well designed or the interpretation is valid.
Method / statistics	Whether declared methodological assumptions and checks are satisfied.	That the scientific question is valuable or fully answered.
Formal verification	A property of an explicit formal object under stated rules and assumptions.	That the formal object faithfully captures the empirical or natural-language question.
Novelty / prior art	What appears known or different within documented search coverage.	Universal novelty or scientific significance.

Evaluation	What it can establish	What it does not establish by itself
Human / domain review	Whether qualified people judge evidence, explanation, and scope adequate for a decision.	Infallible truth or permanent consensus.

The table is deliberately explicit about what each evaluator does not establish. This prevents a verified property from spreading its authority across the whole scientific chain. A proof assistant verifies a formal statement, not the correspondence between that statement and a natural-language research question. A successful reproduction confirms that a procedure can be rerun, not that the interpretation is important. A novelty search can support a claim about prior art coverage, not decide scientific value.

Evaluation diversity matters more than evaluator count. Three LLM critics with similar prompts may create an appearance of consensus while sharing the same blind spots. Greater independence can come from different modalities: execution, formal verification, statistical tests, retrieval, domain knowledge, and human review. The project should know when evaluators depend on the same source, model family, or generated intermediate artefact so that correlated confirmation is not mistaken for independent evidence.

Evaluators themselves need qualification. A citation checker should be tested on supported and unsupported claim-source pairs. A reproduction role should be tested on reproducible and intentionally broken packages. A statistical reviewer can be evaluated on injected methodological faults. A novelty system can use historical cut-offs to test whether it retrieves prior art that existed at the time. Evaluator version and calibration become part of the evaluation object.

The objective is not to automate scientific acceptance. It is to create a structured review case that makes expert judgment

cheaper and better informed. Machines should remove routine work, expose evidence, and identify uncertainty. Humans should spend attention on questions where interpretation, significance, values, and responsibility matter. Evaluation becomes infrastructure for judgment rather than an artificial substitute for the community that owns the judgment.

6.2 From candidate result to community knowledge

A candidate result becomes scientific knowledge through several distinct stages: evidence and verification, explanation and human understanding, review and replication, community acceptance, and eventual canonicalisation. AI can support every stage, but the authority of one stage should not be borrowed by another.

The simplest picture of research is a question followed by an answer. That picture is useful for demonstrations and dangerous for infrastructure. Between a plausible result and a result that a field can rely on lies a chain of work that is often more expensive than generating the candidate itself. Tao reconstructs this chain for mathematics, and the same structure appears throughout science under different names.

A candidate is first connected to evidence. In mathematics this may be a proof object or expert verification. In empirical science it may be observations, calibration, statistical analysis, replication, or triangulation. In software it may be executable code and tests. In qualitative research it may involve source grounding, methodological transparency, and serious engagement with alternative interpretations. The infrastructure cannot impose one universal verifier, but it can make the type and scope of verification explicit.

Explanation is a separate stage. A result can be correct and still poorly understood. Tao notes that machine-generated exposition

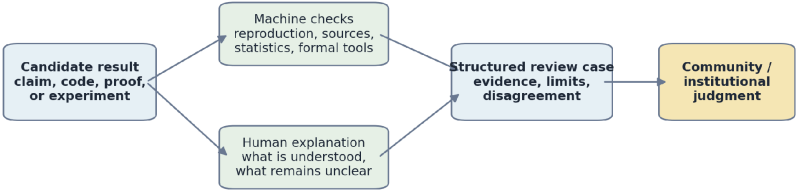
can smooth away the natural friction that tells an expert where the hard part lies. A good research environment should preserve the difficult step, failed routes, unresolved opacity, and the places where expert disagreement remained. Explanation should reveal the structure of understanding rather than merely produce elegant prose.

Review and replication then expose the result to independent challenge. Other researchers ask whether the evidence is adequate, whether the method transfers, whether the interpretation overreaches, and whether the result matters. The process can change the project itself. A reviewer may discover a missing control. A replication failure may reveal a boundary condition. A seminar question may expose that the team cannot explain a dependency it previously treated as routine.

Acceptance is a social act. A machine cannot grant itself scientific standing through fluency or self-evaluation. Journals, institutions, teams, standards bodies, and communities make decisions about what they are willing to rely on under particular conditions. These decisions can be wrong and can later be revised. Their importance lies in assigning scoped authority and responsibility.

Canonicalisation is slower still. A result becomes part of the field's normal knowledge when it is connected to neighbouring ideas, restated in useful generality, taught, reused, incorporated into standards or reference implementations, and perhaps replaced by a clearer formulation. AI may assist comparison, dependency mapping, and explanation, but no model can make itself canonical by declaring consensus.

Evaluation prepares a review case; it does not manufacture scientific authority



Correctness, reproducibility, explanation, significance, and acceptance remain distinct statuses.

Figure 6. Evaluation prepares a review case; it does not manufacture scientific authority.

The diagram shows why evaluation should produce a review case rather than a verdict. Mechanical checks and human explanation meet before community or institutional judgment. This architecture conserves expert attention without turning automation into authority. It also permits a formally or computationally strong result to remain provisional if the human team cannot yet explain what it means.

Table 12. Candidate output becomes shared knowledge through distinct stages.

Stage	What must be preserved
Candidate	Question, assumptions, tool use, source versions, cost, and explicit provisional status.
Verification and evidence	What was checked, by which method, against which data or formalisation, and what remains outside the check.
Explanation and understanding	Key ideas, difficult steps, unresolved opacity, and the human chain able to explain critical dependencies.
Review and replication	Independent criticism, disagreement, reproductions, robustness tests, and responses.
Acceptance	The scoped human or institutional authority that decided the result could be relied upon.

Stage	What must be preserved
Canonicalisation	Integration into theory, practice, reference forms, teaching, standards, and later revisions.

The table allows the project to represent intermediate statuses that current publication systems often handle poorly. A result can be independently reproduced but still disputed in interpretation. A formally checked proof can be preserved while remaining not yet understood. A method can be accepted for one operational context without becoming a general scientific claim. This granularity prevents mature authority from attaching prematurely to early-stage objects.

A provisional layer is especially valuable in an age of machine abundance. Some verified or suggestive results may be too important to discard and too opaque to publish as mature knowledge. The infrastructure can preserve them with explicit limitations, provenance, and a queue for explanation or further testing. The distinction is between preservation and authority, not between keeping and deleting.

Scientific digestion also includes negative knowledge. A failed experiment, rejected hypothesis, or impossible proof route can save others from repeating work. A field that records only successful outputs wastes the search process. Project-level infrastructure can retain negative and partial results privately and later decide which should become public or canonical. AI can help connect old failures to new contexts when assumptions or methods change.

The deeper design principle is that a project should know which stage a result has reached and what would be required to move it further. That knowledge changes orchestration. A candidate with weak evidence needs experiments, not better prose. A well-supported result that nobody understands needs

explanation and expert discussion, not another generation pass. A mature result that remains isolated may need integration into reference material. The infrastructure allocates work according to the scientific bottleneck rather than the convenience of the current model.

6.3 Review as attention allocation and constructive challenge

Peer review should move from a late gate to a continuous, evidence-centred process that prepares consequential issues for scarce expert attention. Automated review should help improve work and create executable challenges, while the evaluator itself remains reviewable and human authority remains visible.

Traditional peer review begins after a project has already hardened into a manuscript. By then, many decisions are expensive to change. A missing control may require repeating an experiment. A data-processing choice may be difficult to reconstruct. The connection between a figure and its code may have weakened. AI-assisted research makes this timing increasingly wasteful because many checks can occur while the project evolves.

A Research IDE can perform continuous review without continuously interrupting researchers. Citation checks can run when claims change. Reproduction can be attempted when a computational result is promoted. Protocol checks can occur before expensive execution. A change in a dataset can mark dependent claims for reevaluation. Most events can remain in the background until risk, disagreement, or consequence justify human attention.

The central problem becomes triage. Expert attention is scarce, and not every warning deserves the same effort. A claim that many downstream conclusions depend on may deserve deep

review even if it appears routine. A surprising result may justify stronger attempts at disconfirmation. An irreversible laboratory action deserves stronger checks than a private exploratory note. The infrastructure should expose why an issue has been escalated and what evidence the automatic system used.

Triage systems can introduce their own bias. A filter trained on past publication decisions may suppress unconventional work. A novelty detector may favour familiar terminology. A methodological critic may create excessive false positives and train researchers to ignore warnings. For this reason, automatic review should remain auditable, allow appeal, and be sampled for missed cases. The absence of a detected problem should never be silently converted into community acceptance.

Review should also be constructive. The most valuable criticism often changes the question rather than assigns a score. A reviewer may suggest a control that distinguishes two mechanisms. A contradiction search may reveal that two papers use incompatible definitions. A reproduction failure may show that the method depends on an undocumented environment assumption. The review object should therefore connect criticism to claims, methods, evidence, and possible corrective branches.

This turns review into executable work where appropriate. A reviewer can request an independent reproduction, an alternative statistical analysis, or a stronger prior-art search. The project can create a branch to test the criticism and return the result to the review case. Other issues remain conversational because judgment cannot be reduced to execution. The infrastructure supports both without pretending that one replaces the other.

A useful operational concept is the review budget. A project can state how much human time, compute, external replication, or laboratory effort it is willing to spend before changing an object's status. When the budget is exhausted, the system reports what

remains unchecked. Exhaustion is not acceptance. This small distinction is important in high-volume environments because otherwise the practical inability to review becomes a hidden lowering of standards.

Reviewers themselves become visible contributors. Data curation, replication, explanation, error detection, and canonicalisation often receive less credit than headline generation. A contribution record can identify who discovered a missing control, who repaired a method, who reproduced a result, or who integrated a technique into a standard reference. Infrastructure cannot change academic incentives by itself, but it can make this work measurable enough that institutions can choose to reward it.

Automated reviewers must also be treated as untrusted input processors. Manuscripts, repositories, and external artefacts can contain prompt injection, adversarial instructions, malicious code, or strategically misleading citations. The component reading the submission should not inherit authority to change policy, access confidential context, or approve the result. Sandboxing and typed tool interfaces are therefore as relevant to peer review as to coding agents.

The success measure for review automation should be outcome-based. Does it reduce the time experts spend on routine checks? Does it improve detection of consequential errors? Does it make evidence easier to inspect? Does it lower correction cost after revisions? Does it preserve uncertainty honestly? A system that writes eloquent referee reports but does not improve these outcomes has automated the appearance of review rather than its scientific function.

This continuous model also changes the relationship between authors and reviewers. Review no longer needs to be a one-time adversarial exchange in which criticism arrives after the project has become expensive to modify. Reviewers can contribute

earlier to the design of decisive tests, while authors can respond by executing branches whose evidence remains attached to the issue. The record preserves disagreement when it is not resolved, so a later reader can distinguish a criticism that was answered from one that was merely overruled. This makes review more useful as scientific work and less dependent on persuasive prose.

The approach also provides a better place for post-publication correction. A retraction or correction need not be represented only as a notice attached to a PDF. The affected claims can be marked, dependent project objects can be identified, and new evidence can be connected to the original review case. Historical authority remains visible, but current users are not forced to treat an obsolete publication state as the latest scientific state.

Continuous review ultimately changes the temporal structure of publication. A paper can become one milestone in an ongoing epistemic object whose claims remain connected to later replications, corrections, and counterevidence. The historical publication decision remains visible while current authority can change. Scientific knowledge becomes maintainable rather than frozen in a sequence of disconnected PDFs.

Chapter 7 — Project Memory and Project Consciousness

Long projects fail not only because they forget facts but because they forget reasons. Conventional logs preserve too much operational detail and too little scientific meaning; summaries often preserve the conclusion while erasing the alternatives, disagreements, and constraints that made the conclusion intelligible. This chapter develops a layered model of project memory and the more speculative notion of Project Consciousness. The latter is explicitly not a synthetic principal investigator. It is a challengeable interpretation service over versioned history whose purpose is to reconstruct why the project changed, recover abandoned directions when conditions change, and expose gaps in distributed understanding. Its credibility depends on resisting retrospective storytelling and on supporting privacy, selective retention, and competing interpretations of the past.

7.1 Why logs and summaries are not enough

Long research projects lose the reasons behind decisions even when they retain files and logs. Project memory should preserve selected scientific events, alternatives, uncertainty, and reasons for change. A summary is not enough because summaries tend to compress disagreement and dead ends into a clean story.

Every mature research programme develops an invisible layer of knowledge that is difficult to transfer. Experienced participants remember why a method was abandoned, which dataset was distrusted, which reviewer objection changed the design, and which idea was postponed because the required instrument did not exist. Formal reports preserve the final position. Raw logs

preserve too much detail. The intellectually useful middle layer often lives only in people.

This fragility becomes acute when agent activity grows. Future projects may contain hundreds of machine-generated analyses, branches, and intermediate artefacts. No human will inspect every operation. A later researcher looking at the final state may see that a branch was closed but not whether it was refuted, unaffordable, unsafe, or merely lower priority. These reasons have very different implications for whether the branch should ever be reopened.

Ordinary summaries are a poor substitute because compression creates retrospective coherence. A language model asked to summarise a month of project activity may remove hesitation, disagreement, failed routes, and competing explanations. The final narrative can be more elegant than the project was, and therefore less educational. It can also be factually wrong when it infers motives that were never recorded.

Project memory should instead preserve meaningful events and their evidence. An experiment failed quality control. A reviewer objected to an assumption. A branch was paused because of budget. A policy changed. A dataset was superseded. A meeting accepted a new objective. These events are higher-level than raw tool logs but more faithful than a retrospective story. They provide anchors from which explanations can later be reconstructed.

The memory also needs explicit uncertainty. Some reasons are recorded decisions. Others are plausible interpretations. A team may never have written why an idea disappeared. A future memory service should be able to say that the cause is inferred from timing and messages rather than present the inference as fact. Competing interpretations of the same history should be allowed when the evidence does not resolve them.

Retention must be selective. A permanent record of every conversation can inhibit candid thinking, create privacy risk, and overwhelm future retrieval. The project should preserve decision-relevant events, source links, accepted rationales, and important alternatives while allowing low-level operational details or personal comments to expire according to policy. The exact compression strategy is a research problem and will differ across domains.

Table 13. Project memory separates operational detail from durable scientific meaning.

Memory layer	Purpose
Raw operational trace	Forensic detail about model calls, tool events, execution, network actions, and low-level failures.
Recorded scientific events	State changes such as experiments, reviews, decisions, invalidations, and branch transitions.
Project memory	Selected rationales, important alternatives, uncertainty, negative results, and links to source events.
Project Consciousness	Challengeable interpretation of history, context recovery, and resurfacing of potentially relevant abandoned work.

The table separates raw operational trace from scientific memory. This distinction allows the platform to be observable without becoming a surveillance system. A forensic investigator may need detailed logs after an incident. A researcher returning after six months usually needs a compact account of what changed and why, with the ability to drill down into source evidence where authorised.

Project memory is useful even without a sophisticated AI interpreter. A timeline of state changes, branches, evaluations, and decisions can already improve onboarding and correction. It can show which assumptions are untested, which claims depend

on one source, or which experiments were invalidated. These are queries over explicit state and provenance rather than model-generated narratives.

The richer concept of Project Consciousness is built above this foundation. Its role is not to create memory from nothing. It interprets authorised history, notices patterns across channels, and helps the team recover context. If the underlying events are sparse or ambiguous, the service should admit the gap. This dependency is important because otherwise Project Consciousness becomes a poetic name for hallucinated institutional memory.

The value of memory also appears in repeated failure. An organisation with many agents can rediscover the same attractive dead end repeatedly. A search should therefore include internal negative results and rejected branches within the authorised scope. A later system can ask whether the old reason for rejection still applies. The project learns not only from what succeeded but from what it once knew not to pursue.

Memory is therefore a scientific resource, not an administrative by-product. In an output-abundant environment, the ability to remember why work was rejected may save more effort than the ability to generate another proposal. The infrastructure should treat the history of scientific change as part of the project object itself.

7.2 Recorded, interpreted, and counterfactual history

Project Consciousness is a mostly passive service that sits above versioned project memory and below scientific authority. It should distinguish recorded events from interpreted explanations and from counterfactual alternatives, allowing its account of the project to be challenged rather than treated as an oracle.

The term Project Consciousness is intentionally metaphorical. It does not claim that a software component is conscious. It names a function that research organisations increasingly need: a persistent observer capable of reconstructing what the project has lived through and making that history useful to current participants. Most agents are active. They search, calculate, write, plan, or execute. Project Consciousness should be primarily passive and therefore easier to govern.

Project Consciousness sits above recorded history and below scientific authority

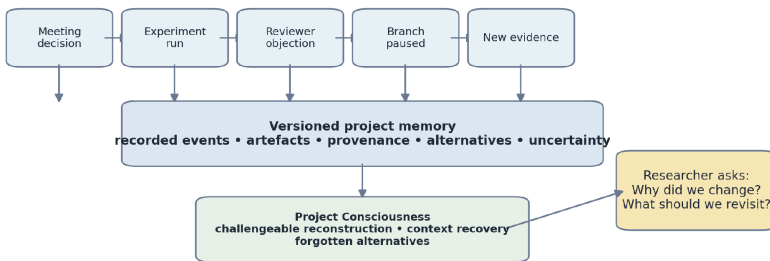


Figure 7. Project Consciousness interprets versioned history while remaining below scientific authority.

The diagram places the service downstream of recorded project memory. Meetings, experiment runs, reviewer objections, branch decisions, and new evidence first become versioned events. Project Consciousness then interprets them and answers longitudinal questions such as why the team changed direction or whether an abandoned idea deserves reconsideration. The service does not commit scientific truth merely because it can tell a coherent story.

A useful implementation should maintain three histories. Recorded history contains observable events with provenance: a branch was created, a protocol changed, an experiment failed, a reviewer rejected a claim, a decision was approved. Interpreted history offers a provisional account of why those events mattered. Counterfactual history preserves serious alternatives

that were not pursued and the conditions under which they might become relevant again.

The distinction resists rationalisation. Human teams naturally rewrite the past around the path that succeeded. An AI trained to generate coherent prose may intensify this tendency by making the final outcome appear inevitable. Project Consciousness should instead preserve uncertainty, dissent, and roads not taken. Its task is not to make the project look wise. It is to keep the project capable of learning from decisions made under limited information.

An abandoned hypothesis illustrates the value. Suppose a team stopped pursuing a mechanism because the available dataset could not distinguish it from a competitor. Years later, a new measurement technique becomes available. A conventional search may find the old documents but not recognise that the reason for abandonment has disappeared. A project-aware memory service can connect the new capability to the old decision and surface the branch as potentially relevant. The suggestion remains a proposal, and researchers decide whether to reopen it.

The same service can notice repeated patterns. Several agents may be rediscovering the same failed path. Multiple conclusions may depend on one weak source. A statistical assumption challenged in an earlier review may have reappeared indirectly. These observations can become attention events without granting the observer authority to resolve them. The value lies in detecting structure across time that no participant may currently hold in working memory.

This separation gives the service an epistemic discipline. Recorded history can be close to factual. Interpreted and counterfactual history should carry confidence, sources, and alternative explanations. If the system infers that a branch was abandoned because a researcher left the team, it should mark

that inference rather than rewrite the historical record as if a formal decision existed.

Project Consciousness also changes explainability. Most explainable AI asks why a model produced one output. Research needs to ask why the project still believes a claim, why an experiment was prioritised, which evidence changed the plan, and what would cause the team to revisit a decision. These are explanations over state and history rather than over one inference. A project-level explanation should point back to objects, events, and authorities that can be inspected.

Privacy is a central design constraint. The observer should not assume access to everything. A consortium may compartmentalise partner data. A meeting may contain personal or sensitive material that should not enter general memory. The system needs explicit blind spots and should be able to say that a conclusion is based only on the events it was authorised to observe. Partial memory is preferable to fictional omniscience.

The concept is falsifiable. A historical reconstruction can be compared with participant interviews and underlying artefacts. Known ambiguities can be injected to test whether the service preserves them. Researchers can evaluate whether revived branches are genuinely relevant or merely semantically similar. If the component produces persuasive but unsupported narratives, its role should be narrowed to retrieval and event organisation. The name Project Consciousness should not protect it from experimental failure.

7.3 Distributed understanding and institutional learning

Scientific understanding in large projects is distributed across people and tools. Infrastructure can help map who understands critical dependencies, where no human can explain a machine-produced component, and how successful or failed processes become

institutional knowledge without pretending that one person must know everything.

Tao's rule of thumb that authors should be able to give a clear expert-level talk about their result is powerful because it makes understanding observable. A formally verified object that nobody can explain is scientifically incomplete even if it deserves to be preserved. In large interdisciplinary projects, however, understanding rarely resides in one mind. A statistician may understand the inference, a laboratory scientist the sample preparation, an engineer the instrument, and a domain expert the scientific interpretation.

The relevant standard is therefore structured collective understanding. The project should be able to assemble an accountable chain in which critical components have qualified human owners, interfaces among specialties are understood, and the team can respond to challenges without appealing to an opaque machine as final authority. Gaps can exist, but they should be visible and should affect the status of the result.

A research environment can maintain an understanding map as a practical extension of project state. It need not grade individuals or monitor every conversation. It can record that a method has an expert explanation, that a formal proof has been reviewed by a designated person, that a machine-generated component remains opaque, or that an interface between two subprojects has not been challenged. Such information helps teams allocate explanation effort before publication or deployment.

WebMeet-like collaborative sessions can make this process active. A result can be discussed while the supporting evidence, code, diagrams, and claim dependencies remain visible. Participants can interrupt, request the source behind a statement, ask another agent to reproduce a calculation, or challenge the weakest assumption. The meeting becomes part of scientific digestion

rather than a communication event whose meaning must later be reconstructed from a transcript.

Project Consciousness can preserve where the difficult questions occurred. A polished manuscript may make every paragraph equally smooth, but the project history may show that one transition required repeated revision or that several experts remained uncomfortable with an assumption. Those friction points are educationally useful. They tell future researchers where understanding was hard won and where current confidence deserves scrutiny.

Institutional learning goes beyond understanding one result. A versioned project history contains repeated patterns of success and correction. Certain review practices may catch errors early. Some methods may repeatedly cause regressions. Some abandoned branches may become valuable when revived under new conditions. The organisation can study these patterns and convert them into process improvements, qualified capabilities, or training material under appropriate governance.

Institutional learning should be distinguished from mere memory accumulation. A large archive is not useful if nobody can identify which patterns transfer. Promotion of procedural memory should therefore resemble capability qualification. A successful workflow becomes a candidate institutional practice, is tested across projects, receives a version and scope, and can be removed when it causes regressions. Local habits should not become global rules simply because they were easy to encode.

The same caution applies to training future models on project histories. Version graphs could teach systems how claims are narrowed, how evidence defeats attractive ideas, and how review improves work. They can also contain confidential ideas, personal criticism, participant data, and context-specific decisions. Permission to use a project for current assistance must remain separate from permission to use it as training data.

Institutional learning requires governance precisely because scientific history is richer and more sensitive than published papers.

A research environment should also measure whether human capability is improving. Researchers should become better able to explain and challenge the work, not gradually dependent on outputs they cannot defend. Junior scientists should be able to learn from failed routes and reviewer objections rather than only from finished prose. The infrastructure succeeds when machine contribution increases the organisation's scientific capacity rather than replacing it with opaque dependency.

The map of understanding can also support succession and resilience. If one critical method is understood by only one person, the project has a human single point of failure even when every artefact is perfectly versioned. The environment can make that dependency visible and encourage a second explanation, a tutorial, or an independent reproduction before the expert leaves. Conversely, it can show where a generated tutorial has not been validated by anyone competent to judge it, preventing abundant explanatory text from being mistaken for transferred expertise.

This perspective suggests a different measure of documentation quality. The question is not how many pages the project has generated, but whether another qualified person can reconstruct the essential reasoning and act safely on it. A concise explanation linked to evidence may be more valuable than a long synthetic report. Project Consciousness can help prepare these explanations, but the project should validate them through actual onboarding and challenge rather than through stylistic evaluation alone.

Distributed understanding therefore connects memory to responsibility. A project can legitimately rely on complex automation without requiring one person to reproduce every

detail. What it cannot do safely is allow the critical chain to disappear into systems that nobody can interrogate. The infrastructure should make the location of understanding and opacity visible enough that authority can be assigned realistically.

Chapter 8 — Direction, Values, and Continuous Governance

A research environment that remembers and evaluates must eventually help with direction and governance, but these are precisely the areas where an apparently intelligent system can become dangerously authoritative. Novelty can become cheap, metrics can become targets, and compliance can become theatre if the infrastructure reduces values to scores. This chapter keeps the relevant dimensions separate. Direction support compares novelty, evidence, understanding, utility, learning value, risk, and cost without pretending that one policy is scientifically universal. Compliance is attached to concrete actions, data flows, rights, and jurisdictions. Human and institutional authority remains visible at the transitions where a project publishes, transfers, spends, commits, or acts in the world.

8.1 Choosing research when novelty is cheap

AI can make unusual combinations and technically distinct proposals abundant. Novelty therefore becomes an insufficient proxy for research value. Direction intelligence should compare evidence, significance, understanding, utility, learning value, risk, cost, and community value while keeping the trade-offs visible and the final decision contestable.

Research institutions have long rewarded novelty because genuinely new results were difficult to produce and often correlated with wider value. A new result might open a field, create a method, explain a phenomenon, or demonstrate a capability. AI weakens this correlation. A system can produce endless unfamiliar combinations, new terminology, and technically distinct implementations. Many may be novel in a

literal sense while few justify the effort required to understand or test them.

Goodhart's law becomes practical here. If novelty is the target, AI can optimise the measurable proxy rather than the scientific purpose. It can combine distant ideas superficially, rename known methods, or search poorly explored regions with little reason to matter. A novelty score can therefore reward the very behaviour that overwhelms reviewers with low-value possibilities.

The response is not to abandon novelty analysis. Researchers still need to know whether an idea is known, whether a method is genuinely different, and whether an investment duplicates prior work. The response is to ask what kind of novelty exists and what it changes. Is there a new fact, mechanism, method, application, synthesis, or failure boundary? Does the result reduce uncertainty, enable a new experiment, simplify a theory, or merely look unfamiliar?

A NOVA-like direction service should therefore be positioned as decision support, not as an oracle. It begins with transparent prior-art search: documented corpora, time boundaries, terminology, and coverage limitations. It then compares the candidate against the project's own history. An idea may look new publicly while repeating an internal failed branch. Conversely, an abandoned internal direction may become newly valuable because a constraint has changed.

Significance is relational. A small improvement can be important if it removes a bottleneck. A dramatic claim can be scientifically shallow if it changes no explanation and enables no further work. Understanding matters because results that reveal mechanisms or compress theory may have long-term value even when they do not immediately improve a benchmark. Utility matters for a declared beneficiary and time horizon. Learning

value matters because an experiment can build a reusable capability even when the main hypothesis fails.

Risk, cost, reversibility, and information value shape priority as well. A cheap experiment that sharply distinguishes two theories may deserve attention before an expensive project with an ambiguous outcome. A risky direction may be worth exploring in simulation but not yet in a laboratory. A project that creates shared infrastructure can be valuable to a community even if it lacks a headline result.

Table 14. Research direction should keep several values visible at once.

Dimension	Question the infrastructure should help researchers examine
Novelty	What is genuinely different within a documented corpus, time boundary, and conceptual comparison?
Evidence	How strongly is the candidate supported, and what observation would disconfirm it?
Understanding	Does it reveal a mechanism, simplify a theory, or connect previously separate knowledge?
Utility	For whom, for what purpose, and over what time horizon could it matter?
Learning value	What capability, data, method, or human expertise would the work create even if the main hypothesis fails?
Risk and cost	What resources and consequences are involved, how reversible are the actions, and which uncertainties dominate?
Community value	Does the direction improve review, teaching, shared infrastructure, or the ability of others to build on the work?

The table keeps these dimensions separate so that an institution can apply its own policy without pretending the policy is scientifically universal. A public funder, pharmaceutical company, doctoral student, and basic-research institute can make different choices from the same evidence. The infrastructure should expose the trade-offs and record the rationale rather than hide the institution's values inside one model score.

Direction support should also identify what evidence would reverse a decision. A recommendation that cannot say what would change it is difficult to challenge. The system can preserve counterfactual branches: if a key assumption were false, if the expensive experiment were delayed, or if a different population were studied, how would the priority change? These are structured explorations of dependence, not predictions that the future is known.

Project Consciousness makes direction more temporally intelligent. A branch may have been abandoned because an instrument was unavailable, not because the mechanism was implausible. A later capability can change the calculation. An earlier negative result can suggest a more precise hypothesis. The value comes from connecting time-separated information that no current participant may remember together.

The system should never become the invisible agenda setter of science. A model trained on past literature can reproduce the conservatism of the field. A commercial service can privilege research that aligns with its owner's incentives. Metrics can narrow inquiry toward what is easy to score. Direction intelligence should therefore remain advisory, plural, and institutionally configurable. People decide which values matter and accept responsibility for the portfolio.

The deeper scarcity of the AI era may be orientation rather than ideas. A research organisation can generate more options than it can pursue. Infrastructure must help eliminate weak branches,

preserve reasons for rejection, and direct expensive verification toward questions that could change the project. The purpose is not a perfect ranking of research. It is a better decision process with more evidence, less forgotten context, and clearer trade-offs.

8.2 Compliance and rights as action-specific context

Compliance should be a continuous property of the evolving project rather than a form completed before or after research. Privacy, copyright, data rights, ethics, safety, and institutional policy depend on concrete actions, actors, purposes, jurisdictions, and current project state.

AI-assisted research makes normative choices continuously. An agent decides which dataset to read, which external model receives context, whether material is copied, whether an experiment can be run, whether a result can be shared, and whether a human approval is required. Treating compliance as a document written after the interesting work has happened is therefore increasingly unrealistic.

The useful unit is a context case. Who proposes what action, for which purpose, using which data and tools, with which recipient or location, under which policies and rights? The same dataset can be lawful to analyse locally and unlawful to send to an external service. A researcher may have the right to read a paper without the right to reproduce large portions of its expression or train a model on it. A partner may allow computation near protected data while prohibiting export.

A compliance component should compile this context from project state and apply rules whose interpretation is sufficiently stable. It can retrieve relevant normative sources, identify possible conflicts, suggest controls, and block actions that are clearly prohibited under accepted policy. Ambiguous cases

should remain explicit and be escalated to qualified human authority. The component is a context compiler and action gate, not an autonomous lawyer.

Privacy can become part of planning rather than a late restriction. When an agent requests sensitive data, the system can ask whether a smaller view, pseudonymised representation, local computation, aggregate result, synthetic fixture, or privacy-enhancing method would satisfy the scientific purpose. Governance becomes design assistance when it helps the project find a legitimate path instead of merely issuing a rejection.

Copyright and licensing require the same action-specific reasoning. Literature agents can search and extract factual claims while still preserving source identity and usage rights. Generated reports should not inherit unexamined copied expression. Proprietary research can route sensitive tasks to private or local models while preserving the same role contract. Replaceable models and controlled deployment are therefore governance prerequisites as much as engineering preferences.

Ethics and safety extend beyond legal permission. A project can be lawful and still irresponsible. It may waste scarce samples, expose participants to unnecessary risk, create dual-use knowledge, or automate decisions with unexamined social consequences. Some constraints can become hard gates. Others require deliberation. The infrastructure should make the distinction visible rather than produce one green compliance status.

Table 15. *Compliance is an action-specific context case, not a generic model score.*

Element	Meaning
Context case	The proposed action, actor, purpose, data, tools, recipients, location, jurisdiction, and current project state.

Element	Meaning
Normative source	A law, licence, ethics rule, institutional policy, contract, safety rule, scientific norm, or project restriction with identity and version.
Applicability	A reasoned and challengeable determination of which sources, exceptions, and scopes apply.
Control	A technical or procedural measure that changes, routes, limits, documents, or blocks the action.
Authority	The person, institution, or accepted deterministic rule permitted to approve, prohibit, or condition the action.
Evidence	The provenance, approvals, disclosures, tests, and receipts retained so the decision can be reviewed later.

The table shows how a compliance decision becomes reviewable. The source and version of the norm are recorded. Applicability is a reasoned determination rather than a hidden model confidence. Controls modify, route, or block the action. Authority identifies who may approve an exception. Evidence records what was checked and why. If policy changes later, the system can identify which future or historical actions are affected.

Statefulness prevents stale approvals from being reused outside their scope. A consent, licence, or ethics decision applies to a particular purpose and context. A new data source, a change from simulation to physical execution, or a move toward publication may create new obligations. The project can reopen the relevant context case instead of treating one approval as permanent permission.

Multi-institution projects make this especially important. A university, company, hospital, and external laboratory may apply different policies without needing one central compliance engine.

The shared project can exchange the evidence each party needs while each institution retains its own authority. The infrastructure supports interoperability by preserving the context and provenance of decisions rather than forcing normative consensus.

A good compliance system should minimise bureaucratic friction. It should infer routine context, reuse previously accepted evidence, and ask focused questions only when a relevant condition changes. The goal is timely governance at the boundary of action. If the system floods researchers with generic warnings, they will route around it and the architecture will fail socially even if its rules are technically correct.

Compliance therefore belongs inside the scientific control plane because rights and safety can determine which research actions are legitimate. It is not an external paperwork service. At the same time, it should remain subordinate to legitimate human and institutional authority. The system makes consequences visible and enforceable where possible; it does not manufacture ethical certainty from language-model fluency.

8.3 Human authority, disclosure, and scientific responsibility

Machine contribution can become extensive without making responsibility disappear. The infrastructure should distinguish contribution from authorship and authority, generate proportionate disclosure from project history, and ensure that consequential decisions remain attributable to people and institutions.

The more work machines perform, the more tempting it becomes to speak as if the machine made the decision. This language can be convenient and dangerous. A model cannot accept legal liability, defend a claim in a scientific community, own an ethical commitment, or bear the institutional consequences of a harmful

action. Responsibility remains with the people and organisations that deploy the system, define its authority, and accept its outputs.

This principle does not require humans to repeat every low-level operation. In a mature automated laboratory, people may not inspect every instrument command. In a formal proof workflow, a proof assistant may check enormous detail. In a large data pipeline, deterministic controls can enforce routine constraints. Human authority can move upward in abstraction: define standards, qualify systems, inspect anomalies, approve exceptions, and accept conclusions. The boundary should be explicit rather than hidden behind a generic human-in-the-loop label.

Authorship is similarly distinct from contribution. A future paper may include a hypothesis proposed by one model, code produced by another, formal checks from a solver, experiments executed by automation, and interpretation developed by a human team. The formal author list can remain human while the infrastructure records these machine contributions at action-level precision. This prevents humans from claiming unaided credit without allowing responsibility to be displaced onto tools.

Disclosure should therefore emerge from execution history. Authors should not need to reconstruct months of AI use from memory. The project already knows which model roles materially affected the work, which computational resources were used, where automated selection occurred, and how outputs were reviewed. A publication or audit view can transform this record into proportionate human-readable disclosure.

Proportionality matters. Grammar correction is not the same as model-generated proof. Routine autocomplete is not the same as selecting a result from ten thousand generated candidates. A standard numerical library is not the same as an opaque external

service performing the key analysis. The disclosure should identify machine participation that changes how readers should interpret reproducibility, attribution, effort, selection effects, or possible failure modes.

The system should also record gaps. An old artefact may lack reliable provenance. A model's training data may be unknown. A source may be inaccessible. Honest uncertainty about lineage is better than an invented chain. Scientific truthfulness includes admitting when the project cannot reconstruct how something was produced.

Table 16. Authority should rise with consequence rather than with model confidence.

Action class	Typical authority treatment
Read public literature or draft private prose	Usually automatic within role scope and ordinary resource limits.
Run reversible sandbox analysis	May be automatic when data class, compute budget, and tool permissions allow it.
Change accepted scientific state	Requires the project policy for the target object, often independent evaluation and human or institutional authority.
Transfer restricted data or contact external parties	Requires explicit rights and recipient checks, with auditable approval where policy demands it.
Publish, submit, purchase large resources, or make institutional commitments	Higher human or institutional threshold because consequences leave the project.
Control physical equipment or safety-critical processes	Bounded capability plus native interlocks and domain-specific operational or safety authority.

The table illustrates differentiated authority. Creating a draft and publishing a claim are different acts. Running a sandboxed analysis and transferring restricted data are different acts.

Proposing an experiment and controlling physical equipment are different acts. The system can grant broad autonomy for reversible low-risk work while retaining explicit human or institutional gates for consequential transitions.

Authority should be revocable. A role that performs reliably can gain a broader operating envelope. A regression, incident, or change in context can narrow it again. The project should record which version of a capability had which authority at the time of an action. This matters for audit and for understanding why historical behaviour was permitted.

The same clarity improves incident response. If a tool is compromised, the organisation can identify actions performed through it, affected scientific objects, and the authorities that accepted downstream results. If a human override bypassed a warning, the decision remains visible. Accountability becomes a property of the system rather than a retrospective search through logs and email.

Human authority is not a magic guarantee. People can be wrong, biased, careless, or institutionally constrained. The point is not that human judgment is infallible. The point is that scientific and social authority must be located where challenge, responsibility, and revision are possible. Machines can provide evidence and execution; communities and institutions decide what status they are willing to grant and answer for the consequences.

This division of labour is compatible with very high levels of automation. A project can delegate thousands of routine operations while remaining human-governed at the level of values, standards, risk, and acceptance. The infrastructure makes that distinction operational so that increasing capability does not quietly become increasing unaccountable power.

Chapter 9 — Grounded and Hybrid Research Intelligence

The previous chapters have repeatedly separated plausible language from stronger forms of evidence. This chapter makes that principle architectural. Grounding is treated as a property of the whole research system, not as a flattering label for a model that cites sources. Neural models, symbolic and deterministic mechanisms, formal solvers, databases, domain tools, and human judgment each contribute different strengths and different guarantee boundaries. The infrastructure should compose them without allowing one component's assurance to leak into another. The chapter also returns to the human interface: a collaborative Research IDE should expose claims, branches, provenance, contradictions, and pending judgments as scientific views, while preserving the familiar documents, conversations, code, and meetings through which researchers actually work.

9.1 Grounding as a system property

A language model is not grounded merely because it cites sources or invokes tools. Grounding is a property of the whole research system when claims and actions remain traceable through evidence, versions, methods, execution, and accountable authority. Strong guarantees should remain attached to the specific layer that earned them.

The term grounded is frequently used as if it described a tone of voice. A model appears grounded when it cites a paper, sounds cautious, or calls a calculator. Scientific grounding is more demanding. A claim is grounded when the route from the claim to the evidence it depends on is explicit enough to inspect and when the transformations along that route have appropriate guarantees. Tool use can help, but a system can retrieve the

wrong paper, execute the wrong code, or formalise the wrong question while producing a beautifully documented result.

Tao's criticism of generative AI is useful because it identifies the structural risk. A generative model is trained to produce an output that looks appropriate under its objective. It can therefore exploit the gap between the visible proxy and the underlying property that the proxy is supposed to indicate. A proof-like answer can look persuasive without being valid. A review can sound expert without checking the evidence. A literature summary can sound comprehensive while searching an inadequate corpus. Fluency is not a scientific guarantee.

The infrastructure proposed here treats model outputs as proposals until other parts of the system connect them to stronger evidence. A citation can be linked to a source span. A computation can be executed in a known environment. A formal property can be checked by a solver. An instrument reading can be tied to sample identity and calibration. A human or institutional authority can decide whether the assembled evidence is adequate for the intended status. Grounding emerges from this chain rather than from one component's self-description.

This still does not make the project infallible. Sources can be wrong, instruments can drift, formal assumptions can be inappropriate, and humans can accept weak evidence. A grounded system is not a system that cannot fail. It is a system in which important failure pathways remain visible enough that they can be challenged, localised, and corrected. Trust comes from reconstructable contact with evidence and authority, not from the absence of uncertainty.

Formal verification illustrates the boundary especially clearly. A proof assistant or solver can establish a property under explicit rules with extraordinary reliability. That guarantee applies to the formal statement it receives. It does not automatically establish

that the statement faithfully expresses the research question, that empirical premises are true, or that the result matters scientifically. Strong verification therefore increases the importance of preserving the translation from natural-language intent to formal object.

A similar boundary appears in statistics. A tested library can compute the requested estimate correctly. The project must still know whether the study design justifies the estimate and whether the interpretation exceeds the evidence. In simulation, a numerical solver can operate correctly while the model represents the wrong mechanism. In security, a model checker can verify an abstraction that omits the relevant physical failure. The infrastructure should display these guarantee boundaries rather than merge them into a single trust score.

Table 17. *Grounding is a chain of accountable transformations.*

Layer	Question that must remain answerable
Evidence source	What observation, source, dataset, instrument, or formal premise anchors the work?
Project state and provenance	Which versions, relations, uncertainty, and rights define the current context?
Reasoning and methods	Which model, program, solver, statistical method, or transformation produced the intermediate result?
Evaluation	What was independently checked, by which method, and what remained outside that check?
Authority	Who or what was entitled to accept the result for the intended scientific or external consequence?

The table presents grounding as a chain of accountable transformations. Weakness at one step does not erase strength at another, but strength should not leak across the boundary. A

formally verified component can coexist with an uncertain formalisation. A complete provenance record can coexist with weak evidence. A human acceptance decision can coexist with an unresolved methodological objection. The project remains scientifically useful because these states are distinguishable.

Grounding is also temporal. A source may be current today and retracted later. A dataset can be corrected. A laboratory calibration can expire. A model can be replaced. The project should know which claims were supported under which versions and which later changes require reevaluation. Static citations alone cannot provide this property; versioned evidence relations can.

The system should also avoid over-grounding through information overload. Returning every available source and log to every agent does not make work more reliable. It can obscure the relevant evidence. The state layer acts as a context compiler that selects the authorised, task-relevant slice while preserving provenance. Exact identity and structured relations complement semantic retrieval. Grounding therefore requires disciplined context, not maximal context.

The strongest argument for system-level grounding is practical. Future research will contain heterogeneous components with different strengths. No single model needs to become a perfect scientific reasoner if the environment can connect generative interpretation to databases, formal systems, statistical tools, sensors, secure execution, and human judgment. Conversely, the presence of these tools should not become a marketing excuse to call the whole system verified. The infrastructure earns trust by keeping the distinctions visible.

9.2 Neural, symbolic, deterministic, and human roles

Scientific work contains several kinds of problem, so no single AI paradigm should own the entire pipeline. Neural models, symbolic representations, deterministic programs, formal solvers, databases, domain tools, and human communities should be composable while retaining their different guarantees and failure modes.

The debate between symbolic and neural AI is often presented as a contest for a complete architecture of intelligence. Research infrastructure suggests a less dramatic and more useful division of labour. Neural models are strong where language is ambiguous, context is heterogeneous, and candidate representations must be invented. Symbolic and deterministic systems are strong where identity, dependency, constraints, and reproducibility matter. Formal methods provide strong guarantees inside narrow formulations. Humans and institutions contribute purpose, values, theory choice, contested interpretation, and social authority.

This division should be visible in the project state even if the user experiences one coherent interface. A neural model can propose that a paper supports a claim. A deterministic retrieval system can identify the exact source. A formal rule can enforce that restricted data cannot leave a location. A statistical package can compute an estimate. A human can decide that the estimate does not answer the real scientific question. The final conclusion is a composition of unlike operations, not the opinion of one synthetic persona.

Axiologic's research on MRP-VM and SOP Lang is relevant because it explores an intermediate representation between open-ended natural language and low-level execution. The important hypothesis is not that one notation will become the language of science. It is that some recurrent language-mediated

tasks can be decomposed into operations, dependencies, branches, validations, and explicit state so that different reasoning mechanisms can cooperate without hiding their boundaries.

Such an intermediate layer can also support deterministic preference. Provenance capture should not depend on a language model if the runtime already knows the inputs. Access control should be enforced by policy, not by prompting the model to remember a rule. Hashes, environment fingerprints, and identity checks should be conventional software. Language models are most valuable where interpretation or generation is actually required.

Symbolic methods should face the same burden of proof as neural ones. A complex ontology or circuit is not valuable because it is inspectable in principle. It must improve correction, reuse, verification, or coordination enough to justify authoring and maintenance. Progressive formalisation provides the discipline: encode the smallest stable fragment that creates leverage, compare it with simpler mechanisms, and expand only when evidence supports the added structure.

Table 18. Different mechanisms provide different strengths and guarantees.

Mechanism	Natural strength	Important limit
Neural models	Language interpretation, analogy, candidate generation, flexible planning, communication.	Fluency does not guarantee evidence, faithful formalisation, or authority.
Symbolic / deterministic systems	Identity, dependency, rules, reproducible transformations, policy enforcement, stable receipts.	Representations can be brittle, incomplete, and expensive to maintain.

Mechanism	Natural strength	Important limit
Formal solvers	Strong guarantees about explicit formal objects.	Guarantees remain conditional on the formalisation and premises.
Databases and retrieval systems	Exact records, indexed relations, scoped external knowledge, temporal snapshots.	Retrieved material can be incomplete, stale, or scientifically wrong.
Humans and institutions	Purpose, contested meaning, responsibility, values, social acceptance, and theory choice.	Judgment is fallible, scarce, and subject to institutional bias.

The table shows why the architecture is hybrid without becoming conceptually muddy. Each mechanism receives a role compatible with its strengths, and the project records which mechanism performed which operation. A symbolic check does not become community acceptance. A neural interpretation does not become a formal proof. A human approval does not make weak evidence strong. The system retains the semantics of the composition.

Hybrid design also improves replaceability. A role that begins with a large language model can later move toward a smaller specialised model, a program, or a symbolic procedure as the task stabilises. Another role may remain generative because the domain changes continuously. The infrastructure does not force one technical destiny across the project. It allows local optimisation under a stable scientific contract.

The same principle applies to external knowledge. Some information is best retrieved semantically from literature. Other information requires exact database queries. Temporal questions require versioned corpora. Provenance requires identity. A research agent that relies only on vector similarity will eventually confuse related material with authoritative material. A research system that relies only on rigid schemas will miss

conceptual novelty. Hybrid retrieval is therefore another consequence of the scientific object, not a separate AI fashion.

Humans remain part of the technical architecture because meaning and authority are operational requirements. A project specification needs someone or some institution to decide which values matter. A contested formalisation needs expert judgment. A reviewer decides whether an explanation is adequate. A community decides whether a result has entered its shared knowledge. These are not gaps awaiting inevitable automation; they are social functions whose degree of automation must be decided rather than assumed.

The long-term objective is therefore not to make the machine look more like a complete scientist. It is to make the research system behave more like a disciplined scientific institution in which several kinds of intelligence and verification cooperate under explicit responsibility. This is a less anthropomorphic ambition and a more demanding engineering one.

9.3 Collaborative scientific control surfaces

When humans and many agents work in parallel, the interface must become a shared scientific control surface rather than a collection of chat windows. Documents, diagrams, claims, evidence, experiments, and decisions should be manipulable common objects whose state and authority remain visible during collaboration.

The surface of future research infrastructure should remain familiar enough that scientists can work without learning a new formal language. Researchers will still read documents, write code, inspect figures, discuss ideas, run notebooks, and join meetings. The difference is that these interfaces become projections of one project state rather than isolated applications connected only by human memory.

A general copilot remains useful because everyday research is varied. A scientist may search literature, edit a method, ask for code, prepare a figure, and plan a meeting within an hour. The mistake is to let the copilot become the project database. A conversation is excellent for exploration, explanation, and negotiation. Persistent state should change only through visible transitions. When a researcher says that a low-temperature regime should be tested, the system can create a candidate experiment plan and ask whether it should become part of the project rather than treating every utterance as authoritative.

Documents can behave similarly. A manuscript remains readable prose, but claims can link to evidence and project versions. A reviewer can select a sentence and inspect the supporting source or experiment. A figure can link to the run that generated it. If an underlying claim is later weakened, the system can mark the paragraph as stale. The document is still a document; the research meaning is connected underneath.

Real-time collaboration raises a harder interface problem. If ten agents can work at once, a scientist cannot control them through ten independent chats. The human needs a shared surface showing which branches are active, which objects are changing, which evidence has arrived, and which decisions require authority. Agent work should appear as proposals attached to common scientific objects rather than as a stream of messages competing for attention.

WebMeet explores this idea as a room in which researchers and a RoboTeam can discuss and create shared artefacts. The important concept is not videoconferencing with a chatbot. It is the meeting as a scientific control surface. A literature role can place a source beside a claim. A statistical role can expose a sensitivity analysis. A modelling role can update a diagram. A critic can identify an unresolved assumption. Participants can

interrupt, redirect, accept, or reject the work while the project state remains visible.

This interface is particularly relevant to scientific digestion. A polished result can be challenged live. An expert can ask why one step is necessary, request the exact evidence, or ask another participant to explain the mechanism without reading generated prose. The system can record which questions were answered and which exposed gaps. The meeting becomes part of the evidence about understanding, not merely a transcript stored after the fact.

Table 19. A collaborative Research IDE needs epistemic views, not only application windows.

View	Question it helps the team answer
Claim view	What supports or contradicts this claim, what status does it have, and what depends on it?
Branch view	Which alternative hypotheses or methods are active, what assumptions differ, and what would discriminate them?
Provenance view	How was this artefact or conclusion produced, with which versions, tools, and approvals?
Contradiction view	Where do sources, experiments, or interpretations disagree and what context could explain the conflict?
Attention view	Which unresolved issues require human judgment because of consequence, uncertainty, or dependency?
Meeting / control view	Which agents are acting, what objects they are changing, and which proposals await acceptance?

The table describes views rather than dashboards. A claim view shows supporting and contradicting evidence. A branch view

compares alternatives and cost. A provenance view reconstructs how an artefact was produced. An attention view identifies issues requiring human judgment. A contradiction view shows incompatible evidence. Different disciplines will need different visualisations, so the infrastructure should support composable views rather than impose one universal research dashboard.

Progressive disclosure is essential. Researchers should not be forced to inspect graphs and traces constantly. The interface can show a concise summary first, then the relevant evidence, then low-level execution details when needed. This keeps the system usable while retaining the possibility of audit. The formal structure remains latent until it improves action or understanding.

Collaborative control surfaces also make authority socially visible. A principal investigator, external reviewer, data steward, and AI agent should not appear as interchangeable participants. When a decision is accepted, the interface can show which role had authority. When an agent merely suggested a conclusion, the suggestion should not visually resemble institutional commitment. Design becomes part of governance because visual ambiguity can create false authority even when back-end permissions are correct.

The social risks deserve testing. Too many agents can fragment attention. Permanent capture can inhibit informal exploration. AI facilitation can steer discussion through subtle agenda choices. A control surface succeeds only if it improves shared understanding and preserves human deliberation. Experiments should compare ordinary meetings, transcript-enhanced meetings, and agent-assisted rooms on decision reconstruction, hidden disagreement, quality of artefacts, and participant understanding.

The future research workplace may therefore be organised less around opening separate applications and more around

convening a scientific situation. Researchers bring the relevant state into view, invite bounded capabilities, and work through an unresolved question while evidence and decisions remain connected. The surface can feel conversational and human even though the infrastructure beneath it is strongly structured.

Chapter 10 — When the Infrastructure Reaches the Laboratory

The laboratory is where architectural ambiguity becomes materially expensive. A plausible plan can waste samples, damage equipment, create unsafe states, or generate an empirical record that is perfectly traceable and scientifically irrelevant. Extending the control plane into the physical world therefore clarifies the entire proposal. Candidate protocols, approved protocols, machine instructions, observed execution, and scientific interpretation are different objects with different authorities. Safety must remain below the authority of open-ended agents, and autonomy must be local to qualified capabilities. This chapter examines how project state, provenance, scheduling, instrument adapters, evidence capture, and human oversight can form a supervisory layer over heterogeneous laboratories without inventing a universal robotic scientist.

10.1 Proposal, approval, execution, observation, interpretation

The laboratory makes the architecture physically consequential. A plausible hypothesis is not an experiment, an approved protocol is not a machine instruction, a machine instruction is not an observation, and an observation is not a scientific interpretation. Future laboratory infrastructure must preserve these boundaries explicitly.

Digital research can hide weak distinctions because the consequences of a mistaken plan are often reversible. An automated laboratory cannot rely on the same ambiguity. A

planning error can waste scarce material, damage equipment, create unsafe states, or generate a false empirical record. The move from digital agents to physical action therefore clarifies the entire thesis of the book: proposal, authority, execution, evidence, and interpretation are different scientific objects.

AI may create an abundance of laboratory proposals long before physical experimentation becomes cheap. A model can generate candidate molecules, parameter combinations, or protocols faster than instruments can test them. Samples remain limited. Instruments are shared. Biological systems have irreducible time scales. Safety and ethical constraints can make some actions impossible. The scarce resource becomes reliable contact with the world.

A laboratory transaction should therefore begin from a scientific objective and an experiment plan. The plan explains what uncertainty the experiment is intended to reduce, which variables may change, which controls are required, and what result would affect the project state. A scientifically weak plan should not become acceptable merely because it is technically executable.

Approval then operates on several dimensions. Scientific authority asks whether the experiment is worth doing. Operational authority asks whether the required equipment and resources are available. Safety authority determines whether the action is physically permissible. Compliance authority considers data, materials, participants, and dual-use implications. Financial authority may constrain expensive resources. These decisions can belong to different roles and should not be collapsed into one approve button behind the scenes.

The approved protocol must then be translated into executable machine instructions. This translation is itself a possible failure point. A robot controller may execute its commands perfectly while the commands omit a condition that was implicit in the

human protocol. The project should therefore preserve the approved protocol, the executable instruction set, and evidence that the mapping was checked for the relevant class of operation.

Execution produces observations, not conclusions. Instrument state, calibration, sample identity, software version, environmental metadata, protocol deviations, and quality checks should accompany the measurement. Where raw data volumes are large, the project can preserve signed references and enough metadata to reconstruct the relation without loading every byte into active scientific state.

Interpretation occurs only after the observation survives relevant quality and methodological checks. A failed instrument run is not a negative scientific result. A quality-control rejection is not evidence against the hypothesis. A successful measurement may still be scientifically inconclusive. The infrastructure should convert low-level execution into semantic events such as measurement passed quality control, protocol deviated, replicate confirmed, or hypothesis weakened.

These boundaries deserve repetition because automated laboratories make category errors expensive. Each transition has its own validation and authority. The architecture avoids the seductive but unsafe idea that a sufficiently capable planning model can own the whole loop. Even very advanced models benefit from lower-level deterministic controls and explicit state because physical systems fail in ways that language models do not predict reliably.

Negative results deserve special treatment. High-throughput systems can turn failed runs into operational noise, yet a scientifically informative failure may be the most valuable output of a campaign. The project should distinguish execution error from evidence that a hypothesis was wrong and preserve both. Future agents can then avoid repeating known dead ends and learn which conditions made the failure informative.

The laboratory also exposes tacit knowledge. Human protocols often rely on skills and local conventions that are not written down because trained people know them. Machine execution forces those assumptions to surface. Progressive formalisation is therefore especially useful here: the protocol can become more explicit as automation increases, while ambiguous or high-risk steps remain under human control until evidence supports stronger automation.

This separation also gives the laboratory a clean failure model. If a high-level planning service disappears, the current experiment can finish safely or stop according to the local controller. If an adapter misreports an instrument capability, the execution layer can reject the request before a physical command is issued. If a sensor becomes inconsistent with another sensor, the run can be marked as requiring review rather than allowing a model to smooth the discrepancy into a plausible explanation. The system should be designed around abnormal conditions because real laboratories are defined as much by failures, delays, contamination, and maintenance as by successful nominal runs.

The same discipline makes qualification transferable. A laboratory capability can specify the instrument family, calibration assumptions, permitted parameter range, safety envelope, and evidence required after execution. A new site can then decide whether to accept the qualification, repeat local tests, or narrow the authority. This provides a path toward interoperable automated science without pretending that heterogeneous laboratories are identical.

A general research environment should not replace native laboratory systems. Instruments and robots have long lifetimes, vendor-specific APIs, real-time requirements, and hard safety mechanisms. The Research IDE should operate as a supervisory scientific layer above them, using adapters and capability contracts. The physical control system remains responsible for

safe real-time behaviour; the research layer preserves scientific intent, evidence, and authority.

10.2 Layered laboratory control and evidence capture

A trustworthy automated laboratory resembles a layered control system rather than one autonomous scientist. Scientific planning, operational scheduling, safety, instrument execution, evidence capture, and interpretation should remain separable so that autonomy can grow without allowing open-ended models to override hard constraints.

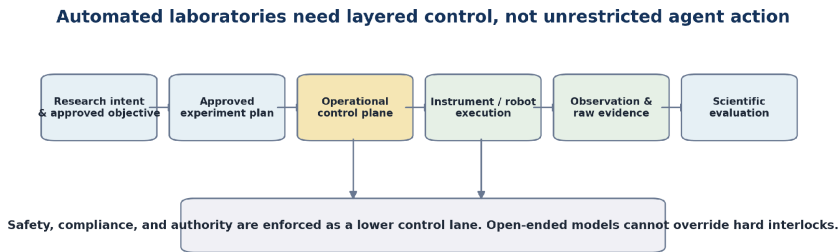


Figure 8. Automated laboratories require layered control rather than unrestricted agent action.

The diagram presents the laboratory path as a sequence with a lower control lane. Research intent becomes an approved experiment plan. An operational control plane schedules resources and binds instrument capabilities. Instruments execute within their native controllers. Observations become raw evidence. Scientific evaluation interprets the evidence relative to the project. Safety, compliance, and authority constrain the entire path and sit below the discretion of open-ended agents where hard guarantees are possible.

This layered design resembles good systems engineering independently of current model limitations. Safety-critical

operations should not depend on a remote language model remaining available. A laboratory should be able to enter a safe state if network connectivity disappears. Hard interlocks should not be overrideable by persuasive text. Real-time control should remain local where timing matters. High-level agents can propose and adapt plans within an approved envelope.

Operational scheduling is another distinct responsibility. Instruments have availability, maintenance states, consumables, and dependencies. A scheduler can manage retries and resource conflicts without deciding the scientific value of the experiment. Direction intelligence can influence priority, but the scheduling mechanism should record why one experiment received scarce capacity and which policy governed the choice.

Evidence capture can be richer than a traditional notebook precisely because automation already knows the machine state. Every measurement can link to sample identity, instrument, calibration, protocol version, software, environmental metadata, and project state. This creates stronger traceability if the information is organised semantically. Raw logs alone are insufficient because millions of machine events do not tell a future scientist which ones changed the interpretation.

The project should therefore compress operational traces into scientific events while preserving drill-down access. A sample was created. Calibration passed. The experiment deviated from protocol. A measurement failed quality control. A replicate confirmed the result. A safety gate blocked a proposed action. These events can participate in project state and Project Consciousness without making the scientific layer a real-time control system.

Digital twins fit into the same architecture as models with explicit validity envelopes. A twin can help an agent test a protocol, predict resource conflicts, or explore failure modes before physical execution. Its predictions are not observations.

The project should record which decisions relied on simulation, what calibration evidence supports the twin, and where reality diverged. A high-fidelity twin can reduce risk; an unqualified twin can convert simulated confidence into physical error.

Table 20. Automated laboratories require several control planes that should not be collapsed.

Control plane	Responsibility
Scientific control	Choose questions, hypotheses, controls, and the information value expected from the next experiment.
Operational control	Schedule resources, bind instrument capabilities, manage dependencies, retries, and recovery.
Safety control	Enforce hard physical and procedural constraints below the authority of open-ended agents.
Evidence capture	Link measurements to sample, instrument, calibration, protocol, software, environment, and project state.
Closed-loop update	Transform validated observations into a new scientific state that can justify, redirect, pause, or block further work.

The table shows why laboratory autonomy should be local to capabilities. A validated pipetting routine may run automatically inside strict bounds. A new synthesis procedure may require direct human review. A routine instrument calibration can be delegated more broadly than a novel experiment involving hazardous materials. The laboratory is not autonomous or non-autonomous as a whole; individual capabilities have authority envelopes earned through evidence.

Multi-site research adds another layer of versioning. Two laboratories may execute nominally identical protocols with different instruments, calibration procedures, and local policies.

The shared project should preserve one protocol identity while making site-specific adaptations explicit. Cross-site reproducibility then becomes a structured comparison rather than a binary claim. If a result transfers only under one instrument family, that dependency becomes scientific knowledge.

Laboratory integration also creates a strong falsification environment for the entire Research IDE thesis. Physical outcomes are less forgiving than fluent prose. Weak project state becomes visible when runs cannot be compared. Bad provenance appears when results cannot be reconstructed. Poor direction intelligence wastes expensive experiments. Overly strict compliance produces false positives that researchers try to bypass. Brittle orchestration encounters equipment failures and asynchronous delays. These pressures make the laboratory a valuable test of whether the architecture creates real control.

The near-term objective should therefore be modest but serious: integrate a supervisory layer with one or two existing automated facilities, preserve their native safety systems, and demonstrate measurable improvement in traceability, recovery, or scientific decision quality. A universal autonomous laboratory is not required to validate the control-plane architecture. A small system that remains understandable under abnormal conditions would be a more meaningful achievement.

10.3 Closed-loop research without a single autonomous scientist

Closed-loop discovery can use machine planning and sequential experiment selection without granting one agent ownership of the scientific loop. Results should update project state through evaluation, and the next experiment should be chosen under explicit values, budgets, controls, and human authority appropriate to the domain.

Closed-loop research is attractive because one experiment can inform the next automatically. Active learning, Bayesian optimisation, and other sequential design methods already choose experiments based on expected information or objective improvement. AI agents can extend the loop by interpreting literature, proposing mechanisms, adapting protocols, and explaining results. The infrastructure should support these capabilities while preserving the distinction between optimisation and scientific judgment.

A numerical optimiser can select the next experiment that maximises a defined objective. The difficult question is whether the objective captures what the project needs. A system can efficiently optimise the wrong proxy. It can also become self-confirming if it repeatedly samples regions where its current model predicts success and treats surprising failures as noise. The living specification should therefore constrain what may be optimised and reserve capacity for controls, replication, and discriminating experiments where scientific value requires them.

Direction intelligence can contribute by asking which uncertainty matters. One experiment may improve a performance metric; another may distinguish two mechanisms; a third may test whether a result is reproducible across conditions. The best next action depends on the project's values and stage. A closed loop should therefore consume the current scientific state rather than only a numerical model of the search space.

Evaluation must remain outside the generation loop. The planning system should not gain authority merely by reporting that its own experiment succeeded. Measurements require quality checks. Interpretations require methodological challenge. Novelty requires independent prior-art search. Consequential conclusions may require human review. The loop closes only when validated evidence changes the project state, not when the planner produces another narrative.

This architecture allows autonomy to grow gradually. In a mature repetitive domain, many cycles can run automatically because the protocol, safety envelope, evaluators, and stopping conditions are well qualified. Humans can supervise policies, anomalies, and changes of regime. In novel or high-risk research, each cycle may require more direct approval. The core project model remains the same while authority changes with evidence.

Several failure modes are hidden by the single autonomous-scientist metaphor. A loop can be operationally successful and scientifically unproductive. It can optimise a biased measure, consume resources on redundant experiments, or create a self-reinforcing interpretation. The state-centred architecture makes these failures inspectable because the objective, evidence, evaluators, and authority are separate objects.

Closed-loop systems should also record why an experiment was chosen from the candidate set. This rationale may later matter more than the raw prediction score. If a branch was selected because it could discriminate between two hypotheses, Project Consciousness can preserve that intent and interpret an unexpected result accordingly. Without the rationale, a high-throughput laboratory can become a producer of measurements whose scientific purpose is difficult to reconstruct.

The laboratory also shows why negative and inconclusive results should update memory. A failed protocol may reveal an operational limit. A null result may eliminate an attractive mechanism. A contradictory replicate may justify more review. The loop should not retry every failure until it obtains a positive outcome. It should classify the failure and let the project decide whether the new state changes direction.

A closed loop also needs explicit pause conditions. Scientific campaigns often enter regimes that were not represented in the original model: an instrument begins drifting, an unexpected

phenotype appears, a new paper changes the prior-art picture, or the optimisation starts exploiting a measurement artefact. The correct response may be to stop the loop and reopen the specification rather than continue searching more efficiently. A control plane that can pause, preserve the current state, and ask for reinterpretation is scientifically stronger than one rewarded only for uninterrupted throughput.

Another requirement is diversity of challenge. A closed-loop system that uses one model family for planning, interpretation, and review can converge rapidly around a shared mistake. Periodic independent reproduction, alternative models, formal checks, or deliberately adversarial human review create friction that protects the loop from self-confirmation. The frequency of such challenge can depend on risk and maturity, but the architecture should make it possible by design.

The long-term vision is therefore not a robot scientist that replaces the institution around science. It is an increasingly automated institution in miniature: memory, roles, permissions, review, safety, versioned protocols, evidence boundaries, and mechanisms for learning from correction. Human contribution may become less operational in some domains, but purpose and accountability remain explicit. The system becomes more autonomous by earning authority in bounded capabilities, not by acquiring a fictional global agency.

Chapter 11 — A Build and Research Programme

A vision of infrastructure is useful only if it can be built, compared with simpler alternatives, and abandoned where evidence does not support it. The final chapter therefore converts the preceding architecture into a research and engineering programme. It identifies a minimum credible substrate, defines acceptance tests for the components, and proposes longitudinal and adversarial evaluations that measure correction locality, provenance, understanding, safe delegation, model replacement, and scientific yield rather than theatrical autonomy. The roadmap is intentionally conditional. Scientific state, evaluators, Project Consciousness, collaborative control surfaces, federation, and laboratory integration should each earn additional complexity and authority through demonstrated value. The programme succeeds by discovering the smallest architecture that preserves scientific control under growing machine contribution.

11.1 A minimum viable research infrastructure

The first credible system should implement a narrow but complete scientific episode: persistent project state, provenance, controlled transactions, bounded execution, evaluation hooks, and a usable workspace. More speculative components should be added only after this substrate demonstrates measurable value on real projects.

The architecture described in this book can easily expand into an unmanageable programme. The defence is to build vertically rather than horizontally. A minimum viable research infrastructure should support one or two real domains end to end instead of implementing superficial versions of every future component. The system should prove that a project-centred

control plane can preserve scientific meaning through actual research work.

The foundation begins with Scientific Project State. The first schema can be intentionally small: questions, claims, evidence, experiments, evaluations, decisions, constraints, and artefacts. Stable identifiers, versions, provenance, access metadata, and relations are sufficient. Domain-specific types can be added through extensions. The primary engineering goal is not semantic completeness but reliable state transition and query.

The second foundation is an event and transaction mechanism. Every persistent change should know which state it started from, who or what proposed it, what execution occurred, what evidence resulted, which evaluations were performed, and which authority committed or rejected the change. This creates a common lifecycle around literature review, computational experiments, document claims, and project decisions.

The third foundation is a controlled runtime. Model calls and tools are executed through capability-scoped interfaces with explicit data access, sandboxes, network controls, resource budgets, and receipts. The runtime should support multiple model providers and local models from the beginning so that replaceability is an architectural property rather than a later migration project.

The fourth foundation is the evaluation envelope. The platform does not need perfect automatic reviewers. It needs a standard way to attach source checks, reproductions, statistical assessments, formal results, novelty comparisons, and human reviews to target objects with declared scope. Even modest evaluators become more useful when their outputs are not confused with final truth.

The fifth foundation is a human-facing workspace. The first users can remain researchers already involved in the development.

They should be able to work through familiar documents, code, meetings, and conversations while the state layer captures meaningful transitions. The interface should expose why a claim is believed, what changed, what is waiting for review, and which branches remain active without demanding constant interaction with graphs.

Table 21. A minimum viable research infrastructure can be assigned concrete engineering acceptance tests.

Component	Minimum credible acceptance test
Scientific state	A real project can version questions, claims, evidence, experiments, evaluations, and decisions without losing native artefacts.
Transaction engine	A complete research episode can distinguish proposal, execution, evidence, review, rejection, and commit.
Controlled runtime	Models and code run with enforceable data, network, secret, and resource scopes and leave usable receipts.
Evaluation envelope	Several independent checks can attach to one object with explicit method, version, uncertainty, and scope.
Capability registry	At least several roles can be swapped between implementations without changing the scientific contract.
Research workspace	Researchers can work through familiar documents, code, meetings, and queries while state transitions remain intelligible.

The table is a build contract rather than a list of aspirations. A team should be able to assign engineering ownership and define acceptance tests for each component. If the state layer cannot support correction propagation, if the runtime cannot enforce capability scopes, or if the workspace cannot present the state

intelligibly, the project has identified a concrete failure rather than merely lacking an impressive demo.

The first end-to-end use case should be computational because it allows rapid falsification. A research question can lead to literature retrieval, a generated or edited analysis, sandbox execution, independent reproduction, claim creation, review, and human acceptance. The entire path can be measured. A second use case can involve multi-institution provenance or a controlled laboratory adapter once the digital substrate is stable.

The architecture should reuse mature infrastructure wherever possible. Git should remain Git. Container runtimes should remain conventional. Object storage should not be reinvented. Existing provenance standards can be adopted selectively. The novel work belongs in the semantic and governance layer that connects scientific objects, agent capabilities, evaluation, and authority. Reinventing every technical substrate would make the research question impossible to isolate.

Axiologic's current lines provide useful starting points without fixing the final design. AssistOS and the ACHILLES IDE can supply the project-centred workspace. WebMeet can test collaborative control. Agent libraries and Ploinky can support bounded capabilities and deployment. MRP-VM and SOP Lang can be evaluated as explicit execution mechanisms where they add value. OpenDSU can contribute to controlled sharing and provenance. Each component should remain replaceable if the experiment shows a better approach.

The minimum system succeeds when researchers can use it for months, not minutes. The project should survive model replacement, participant changes, revisions, failed experiments, and corrections. A new member should reconstruct the current state faster than from ordinary artefacts. A reviewer should trace a claim to evidence. A change in a dependency should identify affected conclusions. These are the mundane but decisive

properties that separate infrastructure from a prototype assistant.

11.2 How to test whether the architecture is actually better

The research programme should compare complete research episodes rather than isolated model outputs. The relevant metrics concern correction locality, provenance, understanding, review effort, recovery of alternatives, safe delegation, model replacement, and scientific yield per unit of scarce human or physical attention.

A system this ambitious must be falsifiable at the architectural level. It is not enough to show that individual components can be implemented. The central claim is that explicit project state and governed agent participation improve sustained research compared with simpler alternatives such as shared repositories plus retrieval-augmented assistants. That claim needs direct comparison.

One experimental design is longitudinal. Two teams work on comparable research tasks. One uses ordinary contemporary tools: documents, repositories, notebooks, chat assistants, and conventional project management. The other uses the structured research environment. Evaluators measure final scientific quality, but also the cost of correction after injected or naturally occurring errors, the ability of new members to reconstruct decisions, the completeness of provenance, and the effort required to replace models or tools.

Correction locality is a particularly strong metric. If one dataset is found to be wrong, how quickly can the team identify affected claims and artefacts? Does the structured environment invalidate only what should be reconsidered, or does its semantic model create excessive false dependencies? If a policy changes, can the system locate actions that depended on the old interpretation?

These episodes test whether explicit state creates practical resilience.

Agent governance can be tested adversarially. Retrieved documents can contain prompt injection. Repositories can contain malicious code. Tools can return corrupted metadata. Agents can be asked to access data outside their role. The evaluation should measure whether typed interfaces, sandboxes, and capability scopes prevent the failure from becoming a project-level incident. It should also measure the usability cost of the controls because security that researchers constantly bypass is not a successful system.

Project Consciousness requires different tests. Historical project records with known turning points can be used to compare its reconstruction with participant interviews and source evidence. Known ambiguities can test whether the service invents coherence. Abandoned ideas can be reintroduced under changed conditions to see whether the system recognises genuine renewed relevance. Privacy experiments can measure whether useful memory can be maintained without retaining excessive conversation.

Evaluation infrastructure should be benchmarked on injected faults and real review outcomes. Citation mismatches, missing baselines, inconsistent sample counts, broken computational packages, statistical errors, misleading causal language, and hidden definition changes can be introduced. Detection, explanation, and final judgment should be scored separately. The central metric is whether experts find consequential issues faster with less routine effort.

Table 22. The architecture should be tested at several scales.

Experimental scale	Evidence sought
Component tests	Parsing, identity, provenance, policy, sandboxing, evaluator fault detection, and deterministic correctness.

Experimental scale	Evidence sought
End-to-end research tasks	Whether components cooperate without losing intent, evidence, status, or authority.
Correction episodes	How locally and accurately the system propagates dataset, source, model, policy, or protocol changes.
Longitudinal projects	Whether explicit memory, qualification, and model replacement create compounding value over months.
Adversarial cases	Prompt injection, malicious code, contaminated retrieval, data exfiltration attempts, and misleading scientific artefacts.
Human studies	Expert review effort, onboarding, shared understanding, ability to explain results, and effects on scientific judgment.
Laboratory trials	Protocol fidelity, safety, evidence capture, abnormal-condition behaviour, and information gained per expensive run.

The table makes the programme intentionally plural. No single benchmark can establish the value of a research infrastructure because the claim concerns sustained scientific control. Local component tests, end-to-end tasks, longitudinal projects, adversarial cases, and human studies reveal different failure modes. A component can be excellent in isolation and harmful when it creates coordination overhead.

The programme should include negative outcomes as legitimate results. Explicit state may create more authoring overhead than correction savings in some domains. Project Consciousness may produce persuasive rationalisations. Symbolic representations may be too costly outside narrow tasks. Multi-agent orchestration may consume more resources than one strong model. Automated review may shift rather than reduce expert labour. A research

programme should narrow or remove components when evidence supports those conclusions.

Human capability should be measured as well. Can researchers explain the result? Do junior scientists learn the critical steps or become dependent on generated prose? Can collaborators identify unresolved disagreements? Does the system preserve the freedom to explore informally? These questions matter because a tool that produces more output while eroding scientific judgment can look productive in short benchmarks and be harmful institutionally.

Physical laboratories provide an especially strong later-stage test. Real instruments expose bad abstractions, weak provenance, and brittle plans. A supervisory prototype can measure protocol fidelity, safety behaviour under abnormal conditions, traceability, resource use, and whether automated experiment selection actually improves information gained per expensive run. Success in this environment would provide strong evidence that the control-plane concept generalises beyond software.

The objective of evaluation is not to prove that the proposed architecture is inevitable. It is to determine which responsibilities genuinely become necessary as machine contribution increases. The final system may be simpler than the vision. That would be progress if the simplification is supported by evidence rather than by neglecting hard problems.

11.3 The path from prototypes to research infrastructure

The build path should move from state and controlled execution toward qualified evaluators, project memory, collaborative control, federation, and laboratory integration. Each layer should receive more authority only after the previous layer demonstrates value and the new layer demonstrates acceptable failure behaviour.

A build path that preserves scientific falsifiability

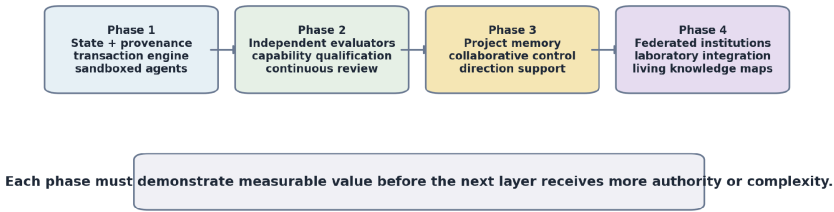


Figure 9. A build path in which each layer must demonstrate value before receiving more complexity or authority.

The roadmap is not a fixed calendar. It is a dependency structure. Phase 1 establishes the substrate: scientific state, provenance, research transactions, sandboxed agents, and a usable workspace. Without this substrate, later concepts such as Project Consciousness have nothing trustworthy to observe and automated laboratories have no stable scientific state to update.

Phase 2 strengthens evaluation and qualification. The platform adds independent evaluators, a capability lifecycle, continuous review hooks, and explicit abstention. This phase tests whether machine-generated work can be challenged systematically without overwhelming researchers. The organisation should begin to accumulate qualified roles and known failure modes rather than only successful demonstrations.

Phase 3 develops richer institutional intelligence. Project Consciousness can reconstruct history. WebMeet-like control surfaces can coordinate human-agent work. Direction support can compare research branches against public literature and internal memory. These components are valuable only if they remain challengeable and improve real decisions. Their authority should therefore remain advisory while evidence accumulates.

Phase 4 extends the architecture across institutional and physical boundaries. Federated research objects, rights-aware provenance, remote capabilities, and automated laboratory adapters test whether the same scientific state can survive heterogeneous organisations and instruments. Hard safety remains local. Shared semantics remain minimal. The objective is interoperability without centralised omniscience.

Table 23. Each layer should earn greater complexity and authority through evidence.

Research direction	Evidence that would justify expansion
Scientific state and transactions	Local correction propagation, stable recovery, useful branching, and lower reconstruction cost than simpler tools.
Governed agents	Role-level qualification, bounded authority, reliable abstention, safe replacement, and lower human correction effort.
Evaluation and digestion	Better error detection, more efficient expert attention, understandable review cases, and clearer provisional status.
Project memory / consciousness	Accurate, challengeable historical reconstruction and useful recovery of context without excessive surveillance.
Collaborative control surfaces	Better shared understanding and scientific artefacts without overload or hidden AI steering.
Federation and laboratories	Rights-aware exchange, safe layered autonomy, faithful execution, robust abnormal-condition behaviour, and complete evidence capture.

The table emphasises progression by evidence. A component should not gain more authority simply because the underlying model becomes more capable. Capability, qualification, and authority remain separate. A stronger model can enter a role in shadow mode. A reliable evaluator can gain broader automatic

use. A laboratory capability can receive a wider operational envelope only after abnormal-condition testing. Regressions should reduce authority again.

Standards should emerge carefully. Premature standardisation can freeze weak abstractions. The programme should first identify stable contracts through real projects: identities, provenance, evaluation envelopes, capability manifests, and research object lifecycles. Where existing standards such as W3C PROV or RO-Crate are sufficient, the system should reuse them. New standards should be proposed only when repeated implementation experience reveals a genuine gap.

The institutional path matters as much as the technical one. Researchers need incentives to record review, replication, and negative results. Organisations need policies for machine contribution and disclosure. Scientific communities need ways to accept provisional objects without confusing them with mature knowledge. The Research IDE can make these practices technically possible, but it cannot unilaterally decide the norms under which they become authoritative.

The long-term product may therefore be a family of research infrastructures rather than one universal platform. A mathematics group may use strong formal verification and expert exposition. A clinical consortium may emphasise privacy, provenance, and regulated authority. A materials laboratory may focus on sample lineage and instrument control. A software research company may prioritise reproducible execution and large-scale agent orchestration. The common layer is the grammar of state, evidence, capability, and authority.

The strongest possible outcome is a new class of research institution in which humans and machines share an explicit evolving model of the work. A more modest outcome would still be important: project-centred environments that make AI-assisted research easier to inspect, reproduce, govern, and

continue across time. The programme does not fail if some speculative components remain human or narrow. It fails if complexity is preserved after evidence shows that it does not improve science.

The path forward should therefore be ambitious in purpose and conservative in authority. Build enough structure to make research episodes measurable. Let machine capability expand where it creates value. Preserve scientific freedom by keeping provisional work cheap. Concentrate controls where actions become authoritative, expensive, irreversible, or external. The architecture then evolves through the same principle it seeks to impose on research: propose, test, evaluate, and commit only what earns stronger status.

Conclusion - The Instrument We Do Not Yet Have

The arrival of powerful research AI does not abolish the old difficulties of science. It changes their relative scarcity. Candidate results, arguments, code, analyses, diagrams, manuscripts, and experimental plans can become abundant. Evidence, expert attention, shared understanding, physical experimentation, responsible judgment, and the ability to integrate results into a living field remain scarce. The bottleneck moves downstream.

This change makes research infrastructure a scientific question. A model can generate an answer. A durable research environment must preserve what the project is trying to discover, which version of the evidence it used, what was executed, what failed, which interpretations remain disputed, who or what evaluated them, and which authority accepted the next consequential step. It must remain coherent when models are replaced, people leave, datasets change, and institutions disagree.

The central architectural proposal is therefore simple enough to state without jargon. Put a persistent Scientific Project State at the centre. Let documents, notebooks, meetings, agents, tools, and laboratories act as interfaces and capabilities around that state. Make meaningful work pass through research transactions that distinguish proposal, execution, evidence, evaluation, and commitment. Give agents bounded project views and separate technical capability from authority. Preserve provenance and versions. Keep evaluation independent and scoped. Let humans and institutions remain responsible for scientific meaning, social acceptance, and high-consequence action.

Once this foundation exists, richer functions become intelligible. Project Consciousness is a challengeable interpretation layer over

versioned history, not a synthetic principal investigator. Direction intelligence compares possibilities and forgotten context, not an oracle that decides what science should value. Compliance compiles concrete action context and gates clear constraints, not an automated source of moral certainty. Neuro-symbolic methods provide inspectable structure where it creates leverage, not a universal replacement for language or human judgment. Automated laboratories extend the same control plane into the physical world under stronger safety boundaries.

Tao's analysis helps explain why such infrastructure is needed. If proof generation becomes cheap, the scientific work does not disappear; verification, exposition, review, digestion, and canonicalisation become more visible as scarce functions. The same principle generalises. A generated hypothesis is not evidence. Executed code is not methodological validity. Reproduced output is not significance. Formal verification is not faithful formalisation. Publication is not canonical knowledge. Scientific institutions advance by converting some candidates through these stages while preserving the ability to correct themselves.

The future Research IDE should therefore be judged less by how autonomous its agents appear and more by whether the project remains scientifically governable as machine contribution grows. Can a new researcher understand why the team changed direction? Can a reviewer trace a claim to evidence? Can a model be replaced without losing project identity? Can a data correction propagate to the right conclusions? Can the system preserve serious alternatives rather than smooth them into consensus? Can it block an unauthorised action without blocking legitimate exploration? Can the team explain what it has accepted and what remains opaque?

These questions also protect the programme from architectural vanity. Explicit state may be too expensive in some domains.

Project Consciousness may hallucinate narratives. Symbolic representations may help only narrow tasks. Multi-agent coordination may create more overhead than value. Continuous review may generate noise. A collaborative control surface may distract rather than clarify. Each component should be tested independently and removed or narrowed when evidence does not support it. The architecture is a research programme, not a creed.

Science has always depended on infrastructures that became almost invisible once they matured: laboratories, libraries, journals, standards, peer review, shared notation, trusted instruments, and professional communities. Those systems were designed for a world in which human production itself imposed substantial friction. AI changes that background assumption. It can make the visible artefacts of scientific work easier to produce than the institutional processes that make those artefacts trustworthy and useful.

Placing a faster generator inside the old workflow is therefore insufficient. The more promising response is to make the implicit architecture of research explicit enough to improve. Evidence should remain connected to claims. Decisions should preserve their rationale. Machine contribution should be attributable without displacing responsibility. Explanation should be valued alongside correctness. Failed branches should remain reusable knowledge. Authority should be differentiated rather than hidden in one assistant. Physical action should remain subordinate to safety and approved scientific intent.

The future scientific instrument is not an oracle. It is a governed environment in which humans, AI agents, formal systems, databases, code, evidence, and laboratories can cooperate while their different roles remain distinguishable. Such an instrument can make science faster without requiring science to become shallower. It can make machine labour abundant while keeping

judgment scarce enough to matter. That is the infrastructure this book argues we should build.

Selected External Sources and Axiologic Research Context

Tao, Terence. Mathematics in the Age of AI. arXiv:2608.16753v1 [math.HO], 17 August 2026. Essay based on the 2026 International Congress of Mathematicians lecture.

European Commission CORDIS. ACHILLES - Human-Centred Machine Learning: Lighter, Clearer, Safer, Grant Agreement 101189689. The project context includes work toward an integrated, specification-oriented environment for trustworthy AI-assisted research.

Axiologic Research. Research and technology programme around AssistOS, the ACHILLES IDE, WebMeet, OpenDSU, agent infrastructure, MRP-VM, SOP Lang, provenance, privacy, and executable-science experiments.

European Commission. European AI in Science Strategy and RAISE: Resource for AI Science in Europe, including work toward automated scientific discovery and AI-enabled research infrastructures.

Lu, C. et al. Towards end-to-end automation of AI research. Nature, 2026.

Gottweis, J. et al. Accelerating scientific discovery with Co-Scientist. Nature, 2026.

Ghareeb, A. E. et al. A multi-agent system for automating scientific discovery. Nature, 2026.

NIST. Artificial Intelligence Risk Management Framework and AI Resource Center, including context-dependent trustworthiness and testing, evaluation, verification, and validation practices.

W3C. PROV family of provenance specifications.

RO-Crate Community. RO-Crate Metadata Specification 1.3, 2026.

Git Project. Pro Git documentation on version control, commits, branching, and recoverable collaborative change.